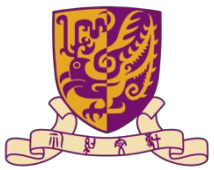
香港中文大學統計學系
Department of Statistics
THE CHINESE UNIVERSITY OF HONG KONG

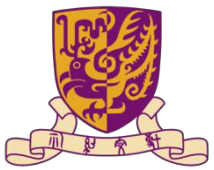
STAT6207
Applied Deep Learning

11 Sept 2026
By Dr. Ming Cheung

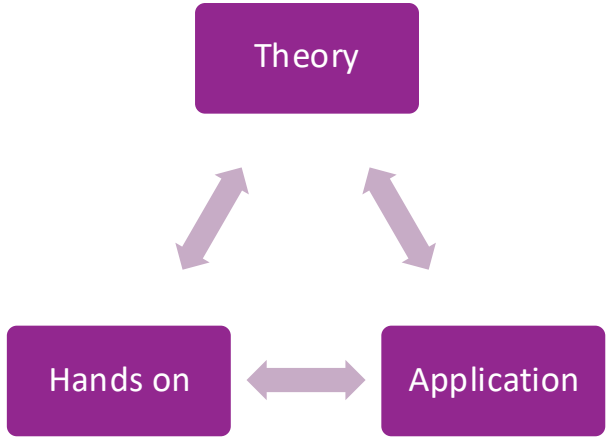


Agenda

- Course Introduction
- What are machine learning
- Supervised Learning and KNN



What is **STAT6207**



1. Learn the **theory** of machine learning and deep learning;

2. Understand machine learning **applications** to solve real problems in generative adversarial networks, large language model, etc.;

3. Gain **hands on** experience through projects and activities using agentic coders and Python libraries, such as OpenCode, PyTorch and Numpy, with projects

Prerequisite/Background

- Mathematics: multivariable calculus, linear algebra and probability
- Coding: OpenCode, Python and NumPy, probability and machine learning.
- Course material and other information will be posted on Blackboard
- Pre-slide for the week will be released every Wednesday



Policy on Generative AI

- You are permitted to use generative AI (e.g., ChatGPT, Claude, Grok, etc.) and the Agentic Coder (i.e., OpenCode), in assignments, programming exams, and projects
- When generative AI is used to produce code, text, or reports, students must transparently declare its use by including an appendix for grading:
  - A shareable link to the conversation(s) with the AI (including all prompts and AI-generated outputs), if web-based AI is used.
  - In case of using agentic coders, the session transcript must be submitted.
  - A detailed description of all modifications made to the AI-generated content.
- Do not use it to skip learning



My expectations in Teaching

- Please ask me questions about the materials, and feel free to ask me any time even during my presentation

- Discussion is part of the learning; you are welcome to discuss your challenges with me during the breaks and after classes. You don't learn to swim by reading about water.

- Agentic Coders and LLM are tools for build, but you should spend the time to understand the concepts and build them

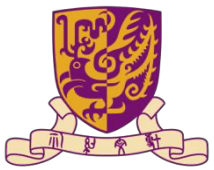


Course Introduction

Type	Description	Weight
Assignments	4 assignments	20%
Examination	A written/programming examination (L7, class time)	30%
Project	A project with a report and a presentation (L14, class time)	50%

6. Grade descriptors

Grade A	Demonstrate excellent knowledge of all the learning outcomes listed from (1) to (3) and able to apply multiple types of knowledge in solving a machine learning problem.
Grade B	Demonstrate excellent knowledge of all of the learning outcomes listed from (1) to (3) and able to apply one type of knowledge in solving a machine learning problem.
Grade C	Demonstrate partial knowledge of the learning outcomes listed from (1) to (3) and can only apply some knowledge in solving a machine learning problem.
Grade D	Demonstrate segmental knowledge of the learning outcomes (1) to (3) and barely demonstrate any knowledge in solving a machine learning problem.
Grade F	Fail to learn about the key topics in learning outcomes (1) to (3) and fail to apply any knowledge in solving a machine learning problem.



Marking Scheme (Final Project)

Format: 1 presentation and 1 report

Data processing and Methods

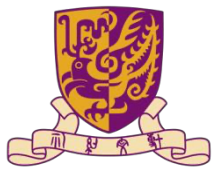
- apply what you have learned or any techniques on the data to achieve the expected results
- state clearly what methodology or algorithms you used and how you used them to process the data in your report

Experimental results

- state clearly your result in your report.
- You use tables, graphs, charts or other visualization methods to demonstrate your conclusion.
- You draw interesting conclusions/insights from your results.

Presentation and Organization

- Your report is organized in a way that is logical and easy for the reader to understand
- The overall language you used in your report should be concise and able to clearly convey your idea.
- Only selected groups will present their works to the guests.



Last Semester Reports

STAT 6107 Final Project Report
Topic 2: Store Sales Prediction
(Active Mind Limited)

Group 3

Chiu Ming YIP
1155172863

Kit Yung LAU
1155169534

Ling Wai LI
1155169344

Nga Man Almen WONG
1155167098

On Yi KO
1155048647

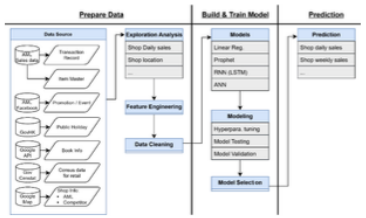
I. Problem Statement

Active Minds Limited (AML) has been selling English Children Books and Educational Toys with physical stores across Hong Kong since 2009. They are facing difficulty predicting customer demand which will impact inventory cost and cash flow. Our project objective is to predict weekly sales for each shop in 2023 by machine learning techniques based on historical transaction and item data. Based on the findings, future sales strategies will also be recommended to help to improve revenue.

II. Machine Learning Architectural Design

The following image shows an overview of the architectural design of the prediction model, processes include: data collection, exploration analysis, feature engineering, data cleaning, model building and final prediction.

Fig. 1. Model Architectural Design



III. Data Preparation and Exploration

TABLE I. Data Table and Sources

		Data Format	
		Structured	*Unstructured*
Data Source	*Internal*	• Transaction data • Item data	• Store address from website • Research on social media platform for promotion detail
	External	• Public holiday calendar • Government Census data • Book info from Google Books API	• Online Competitor Analysis • Site visit to physical stores

A. Internal Data

Source data is the transaction data and item data in 202 and 2022. To further understand customer behavio additional shop information such as shop location, shop size promotion campaign period and its discount level wer solicited from AML social media platform and website.

B. External Data

External data such as the public holiday calendar, retai market data from the Government Census Publication an book info from Google Books API were also sought t understand the market trend and seasonality. Two competitor Bookazine and Kiddyland, were also identified for analyzin shop distribution in the exploratory analysis

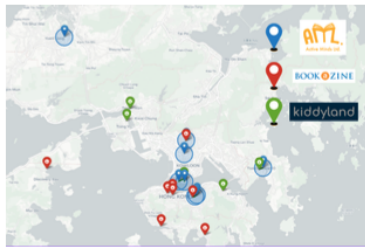
IV. Data Cleansing

Before model building, we streamlined the raw dataset b dropping incomplete or mismatched transaction rows handling exceptions for typhoon/CNY periods, an converting categorical variables such as public holidays grou into numerical dummy variables. The unstructured data wa then transformed into a clean, formatted dataset to trai machine learning models for insight.

V. Exploratory Data Analysis

A. Shop Distribution

Fig. 2. Shop distribution of AML and competitors



AML and competitors also concentrated shops in th prime shopping areas such as Causeway Bay, which suggeste that their target customers are those with high spending powe AML also opened some shops in residential areas like Tseun Kwan O and Yuen Long, where the competition was not a keen as the shopping area.

Product Matching Algorithm with Deep Learning for Competitor Analysis

Wai Ying Chan
Department of Statistics
The Chinese University of Hong Kong
Hong Kong SAR
1155150554@link.cuhk.edu.hk

***Abstract*— This research proposes a product matching algorithm to help Lane Crawford, an international fashion company, understand their competitive landscape by comparing their products with their competitors. The project focuses on women tops and uses pretrained models to extract features from product information, including images and text. Lane Crawford and Farfetch data are collected through web scrapping, and a total of 1,219 Lane Crawford items and 18,453 Farfetch women tops items are obtained. Test data are hand-picked randomly from both websites for evaluation purposes. The report presents the methodology used for data collection, data selection and preprocessing, and feature extraction and weighted similarity. The experiments section evaluates the algorithm's performance using two different models: CLIP [1] and FashionCLIP [2]. Results show that FashionCLIP, a fine-tuned model with 800,000 Farfetch data, achieves a higher accuracy of 90% in finding the correct best match compared to the CLIP model's accuracy of 36%.**

Keywords—fashion, competitor analysis, CLIP, Fashion CLIP, pricing model, product matching, image similarity, text similarity

I. Introduction

In today's competitive market, it is crucial for businesses to understand the competitive landscape to gain a competitive edge. Lane Crawford and Farfetch are international fashion companies that sell modern luxury fashion items, including women's and men's clothing, accessories, and more. They have a mature eCommerce business to sell their items worldwide. To help Lane Crawford understand its competitive landscape, this report proposes a product matching algorithm that uses deep learning to compare their products with their competitors, specifically Farfetch. The algorithm will focus on women's tops and compare pricing information. This report will present the methodology used for the project, the experiments conducted, and the results obtained.

II. Methodology

A. Data Collection

The data collection process involved obtaining data from both Lane Crawford and Farfetch. Lane Crawford provided a dataset in CSV with 13,431 items. As the provided dataset may not include the latest items, making evaluation difficult, web scraping was performed on April 23, 2023, to obtain the latest product details and images from the Lane Crawford website, resulting in 1,219 items collected. Web scraping was also performed on April 22, 2023, to collect Farfetch product data, and 18,453 items (approximately 60% of Farfetch women tops items) were successfully scraped for finding the best match from Farfetch items. To evaluate the algorithm's performance, 42

B. Data Selection and Preprocessing

Lane Crawford and Farfetch have different images (Fig. 1). Lane Crawford has at least two image types for each item, "in" and "ro." Type "ro" is a full-body version in which a model is wearing the product while there may be another product in the image. Type "in" is a zoom-in version in which only the product is clearly shown in the image with less unnecessary information. Hence, image type "in" will be used in this project. Farfetch also has at least two images for each product called "cutout" and "model." Type "model" is similar to type "ro" from Lane Crawford while the models may wear different bottoms in the images. In this case, type "cutout" will be used for comparison.



Fig. 1. Types of Images from Lane Crawford and Farfetch

Besides images, text information is also important for finding the best match. In this project, brand name, product name, composition, and long description will be considered. A new column named "product description" is created by concatenating product name and long description as they provide similar information. Additionally, the leading and trailing spaces of text are removed before being used. Since there are processors for image and text attached to models introduced later [3][4], little data preprocessing is required.

C. Similarity

The image embeddings and text embeddings of the products are computed in advance with different models. Cosine similarity is calculated for each type of embedding pairs. That is, calculate cosine similarity for image embedding pairs, brand name embedding pairs, etc. Weightings are assigned to these similarities in order to

Develop an algorithm for Ztore to automatically index data for efficient dashboard querying to support real-time response

Chan Yuen Lai
Department of Statistic
Chinese University of Hong Kong

Leung Tsz Kin
Department of Statistic
Chinese University of Hong Kong

Ip Blanche Kong Kei
Department of Statistic
Chinese University of Hong Kong

Ng Yat Hei
Department *of Statistic*
Chinese University of Hong Kong

***Abstract* — Speedy query performance is desirable in the era of big data, especially to the e-commerce industry. Developing an algorithm to generate indexes for SQL queries and data columns is important to support the needs on real-time dashboard. 3 Unsupervised models, Association rules, Hierarchical Clustering, and K-means Clustering are examined in this study. This paper provides a comprehensive analysis of which algorithms can effectively cluster columns in the SQL queries and ultimately reduce the speed on retrieving the data. In addition, this study compares the best number of indexes by evaluating the statistical score for the classifiers, overall queries that can be benefited by the extra index created, and the business nature for Ztore. In the end of this paper, a selection chart is suggested to optimize the number of indexes under the constraint of cost, database capacity, and target queries to be benefitted.**

I. Introduction

The objective of this study is to create indexes that enable more efficient dashboard querying for real-time response with Machine Learning method. Clustering SQL queries based on similarity are suggested in much research [1] as the distance between clusters can evaluate the similarity between queries. To achieve this, several data pre-processing work are conducted to the raw datasets, and then apply to various data models. With parameter tuning in each model, number of rules/ clusters is identified. By comparing the result, columns which could be grouped and the number of queries to be benefited are identified. Semantically similar queries are then clustered together and generate a new index for each cluster to comply with Ztore data.

Assumption and setting. 3 assumptions are considered in this study. Different groups of Ztore's SQL are heterogeneous and can be grouped based on similarity. The index rules are separately considered for WHERE and GROUP-BY statement. For the WHERE conditions, 'AND' logic is assumed while 'OR' logic only applied to conflicting filters, i.e., multiple age groups in the query.

II. METHODOLOGY

A. Data Pre-processing

Data cleansing improves accuracy and reliability for the input data for model building. Both SQL and transaction data are cleaned by flattening the JSON list and remove unnecessary values in the raw datasets.

B. Unsupervised Model building

3 Unsupervised and distance-based models are examined in this study, including Hierarchical Clustering, Association rules, and K-means Clustering [3].

each other. It creates a hierarchy of clusters by iteratively merging smaller clusters into larger ones until all data points belong to a single cluster.

Association Rules. A data mining technique to discover patterns or relationship between items in a dataset. The co-occurrences of columns in the SQL queries are concerned.

K-means Clustering. An algorithm used for clustering similar data points together. The goal of K-means clustering is to partition the data points into K groups where k is user-defined. It works by iteratively assigning data to the nearest centroid and recalculating the centroids based on the means of data points assigned. This process continues until the centroid no longer move.

C. Model Comparison and Result Interpretation

Based on the model result, different clusters are identified, and models are compared based on the Silhouette score, the interpretability of data columns in each cluster, and the effective ratio (number of queries benefited / number of indexes created). The final model is selected to group columns and create indexes to speed up the query process.

D. Decision-Making and Business Applications

A selection matrix on utilizing the model result is developed to assist Ztore to make decisions on number of indexes to be created. Suggestions on handling the 'AND' and 'OR' conditions in the queries are considered in the final application on transaction data.

2. Data Pre-Processing

Data processing is conducted to both Ztore SQL query data and transaction data for index creation.

E. SQL Query Data

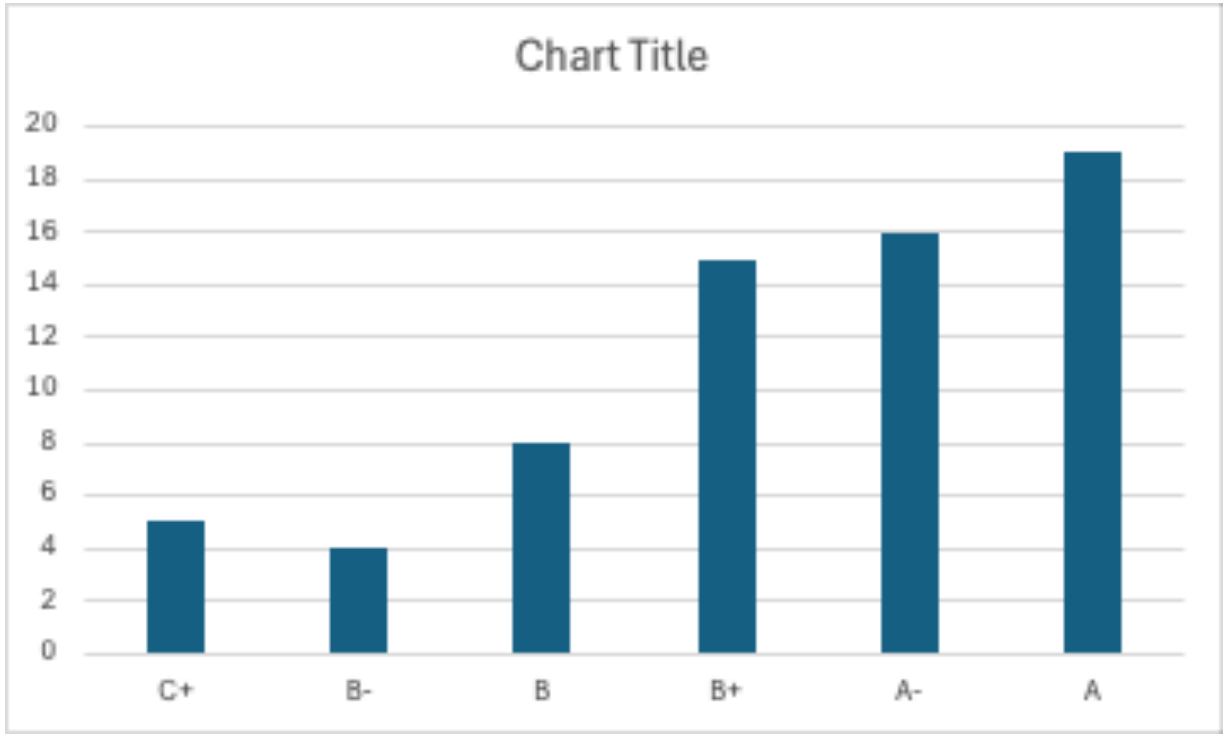
The dataset is used as the input of the model. It consists of 2 major columns, which are WHERE and GROUP-By. Since the original data column contains JSON list in each cell, flattening the list to identify all the combinations are needed. Then, one-hot encoding is conducted to convert the data values to [0,1] to calculate the cosine similarity between each query [2].

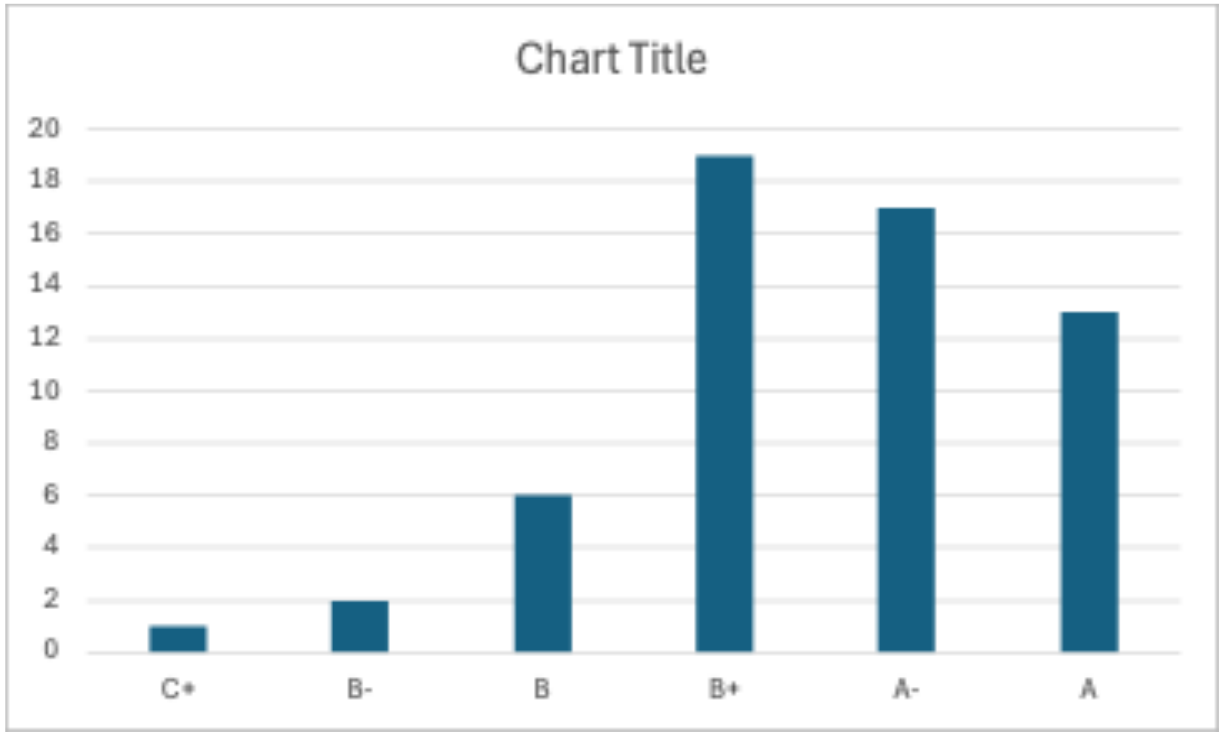
F. Transaction data

Several processing is conducted to the transaction data for index creation.

- Some unnecessary string values in Age, Gender, and District columns are cleaned and reduced to an integer value, i.e., storing 1 in Age Group column instead of storing "age group 1".

Last Semester Grading





Course Topics (To be confirmed)

Lecture	Topics
1-2	ML Foundations (Supervised Learning, KNN, Transfer Learning & ResNet)
3-4	LLM, Architectures & Efficiency (RAG, CoT, Agentic AI, Transformers, Quantization, etc.)
5-6	Computer Vision & Multimodal AI (CNNs, Vision Transformers, Visual Encoders, VLMs & Face Recognition Demos)
7	Examination
8-10	Deep Learning Core & Unsupervised Learning (Embeddings, Autoencoders, Clustering, GNNs, Backprop, Optimizers & Deployment)
11-12	Generative AI, RL & Advanced Paradigms (GANs, Stable Diffusion, Federated Learning & Reinforcement Learning Reward Models)
13	Ethics, Course Review & Future Horizons (Bias, Ethics, Course Review)
14	Project Presentation



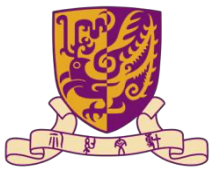
Course Topics compare to 2025

- Added in 2026
  - More on Agentic AI and the use of OpenCode
  - LLM-based clustering
  - KV caches, MoE, quantization & mix-precision
  - Using Local LLMs (e.g., Qwen, Gemma) for deep learning applications



Agenda

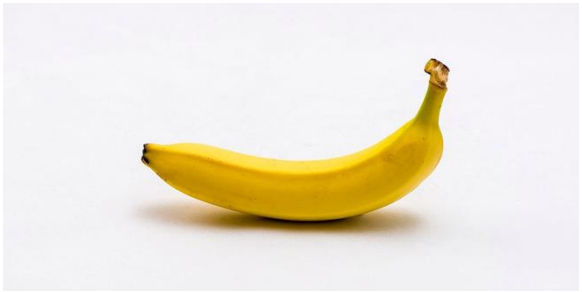
- Course Introduction
- What are machine learning
- Supervised Learning and KNN



What is Machine Learning

- Classification
- Regression

How do you tell if they are bananas or apples?





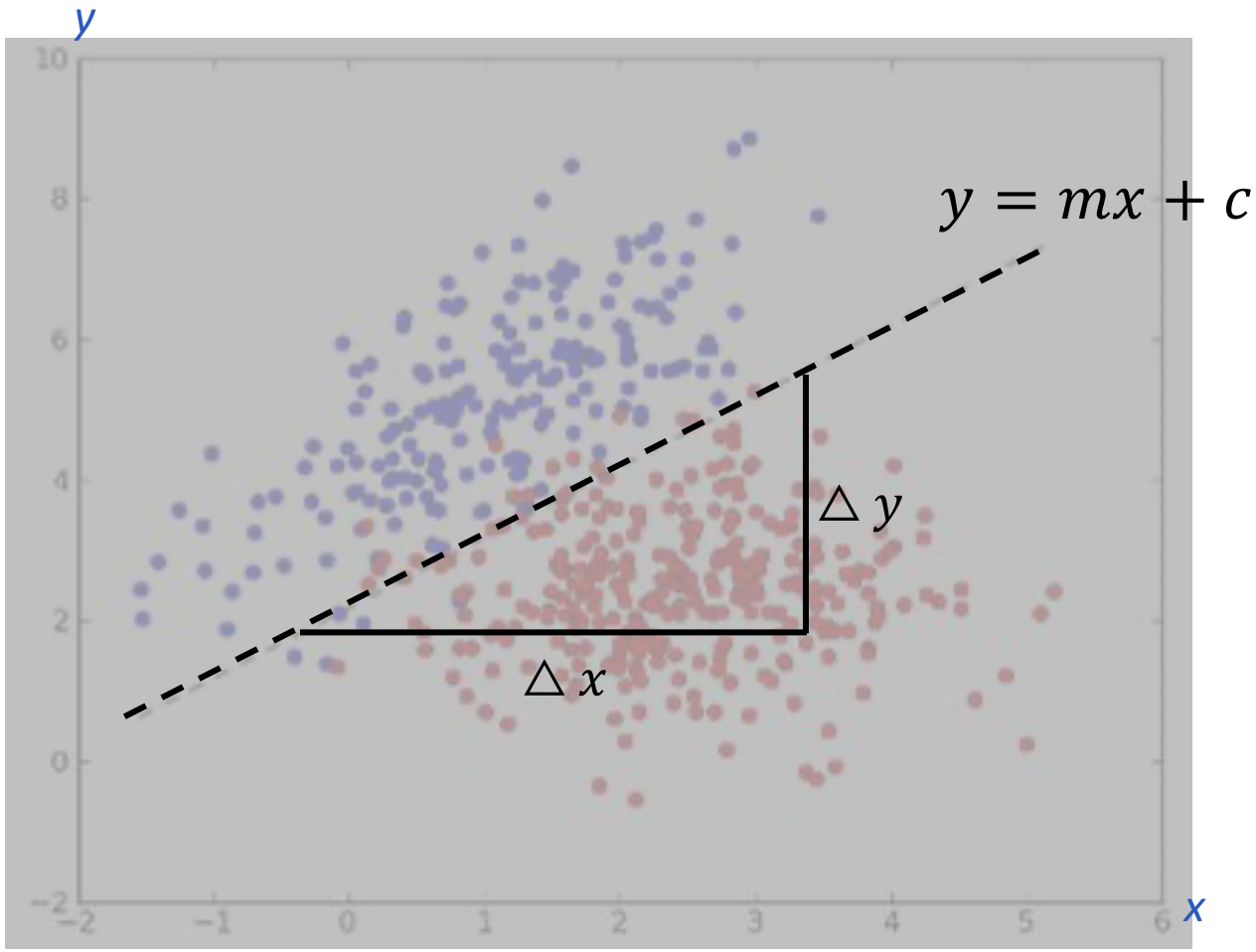
How do you tell if they are dogs or cats?





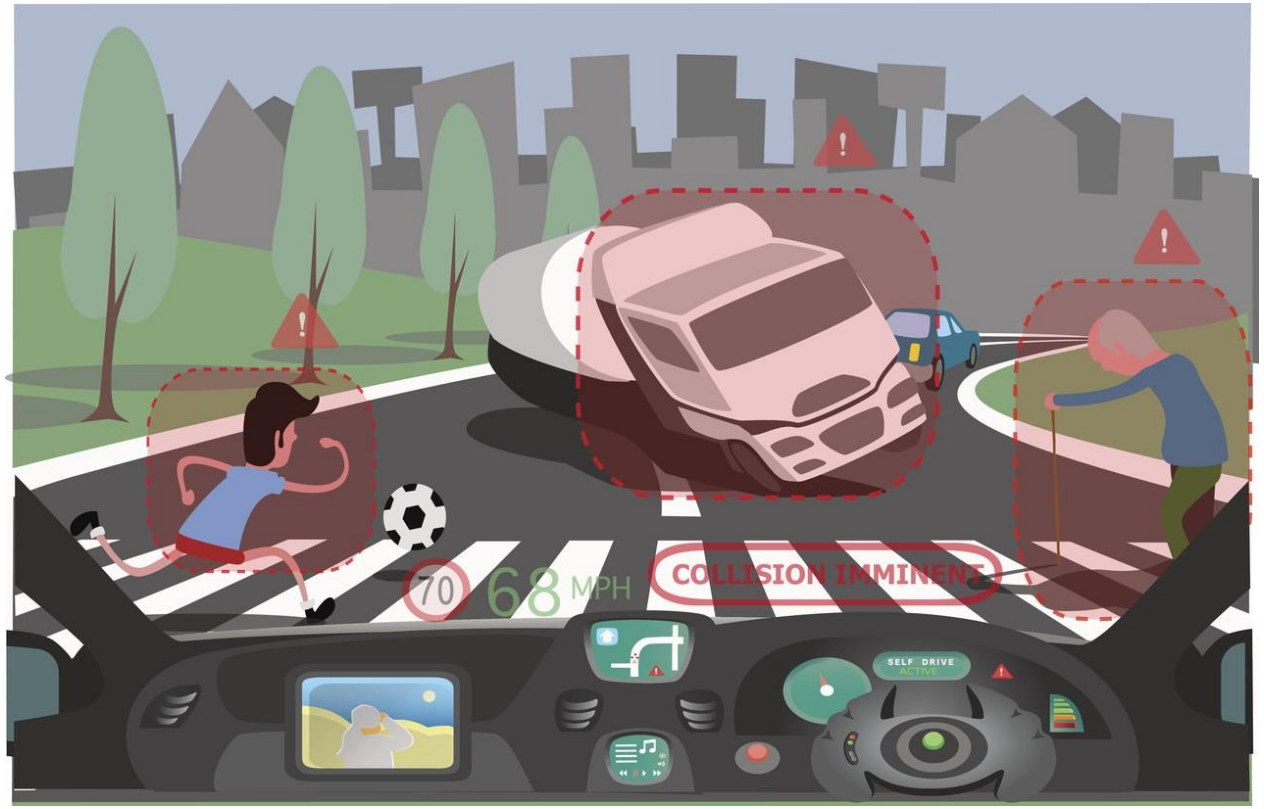
What is Machine Learning - Classification

- Using a straight line equation as the separator.
- Variables: x and y
- Constants: m and c
- On graph:
  - m = gradient
  - c = y-axis intercept



What is Machine Learning

- What should the driver do?



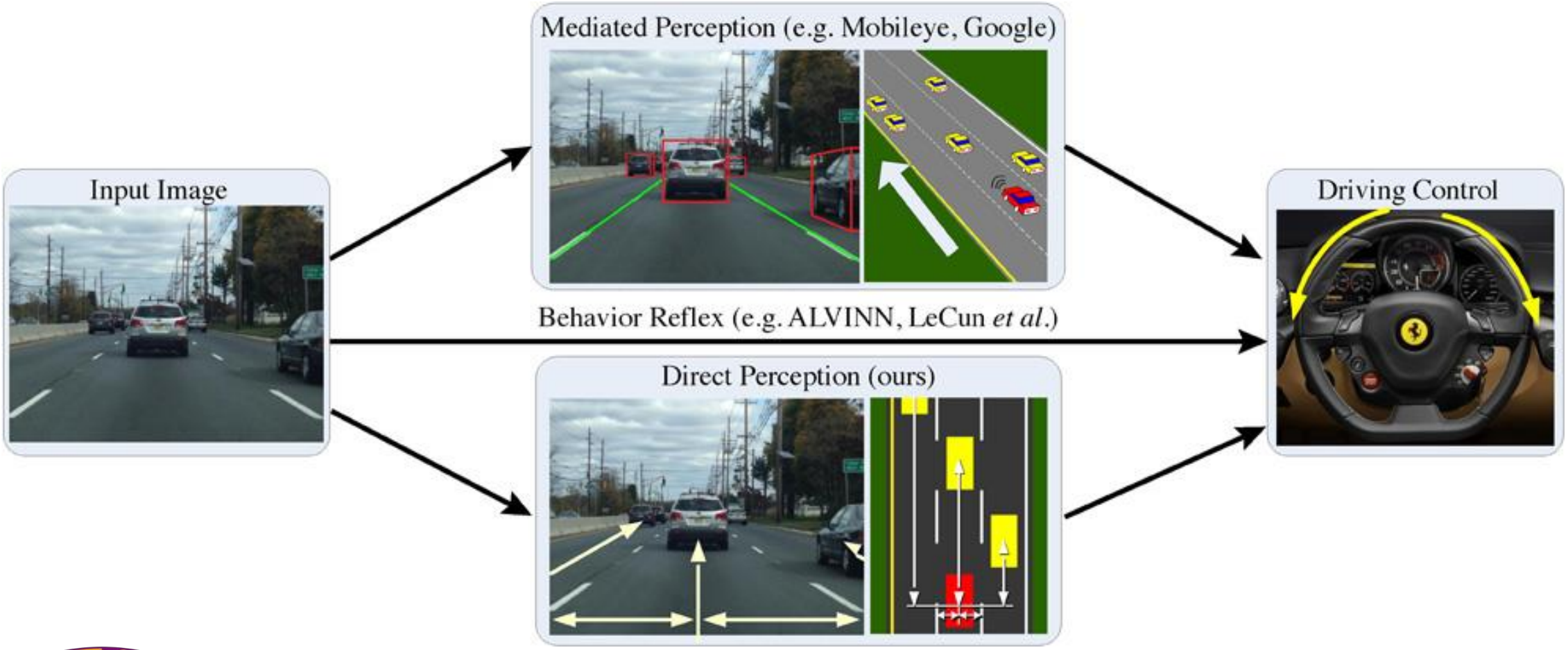
What is Machine Learning - labeling

- Where do you want to car to go?



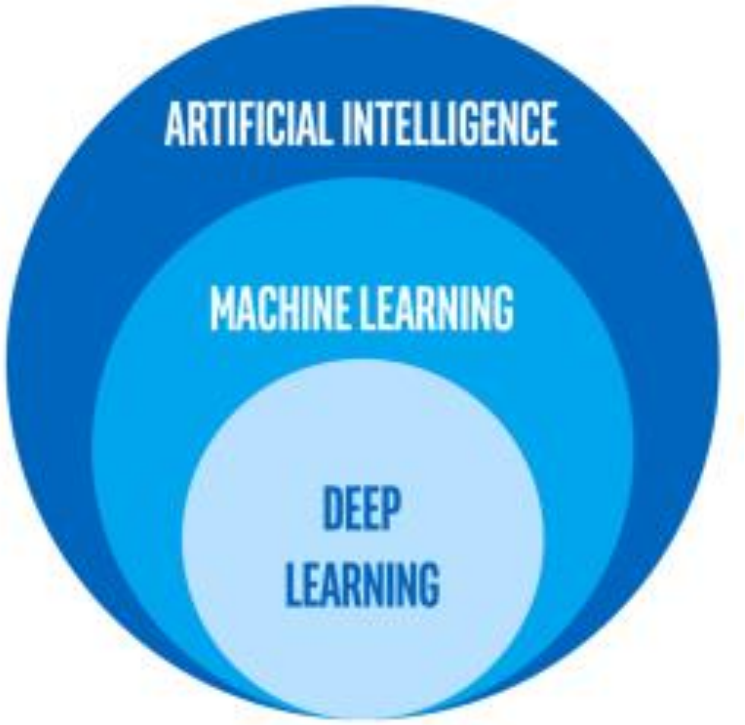


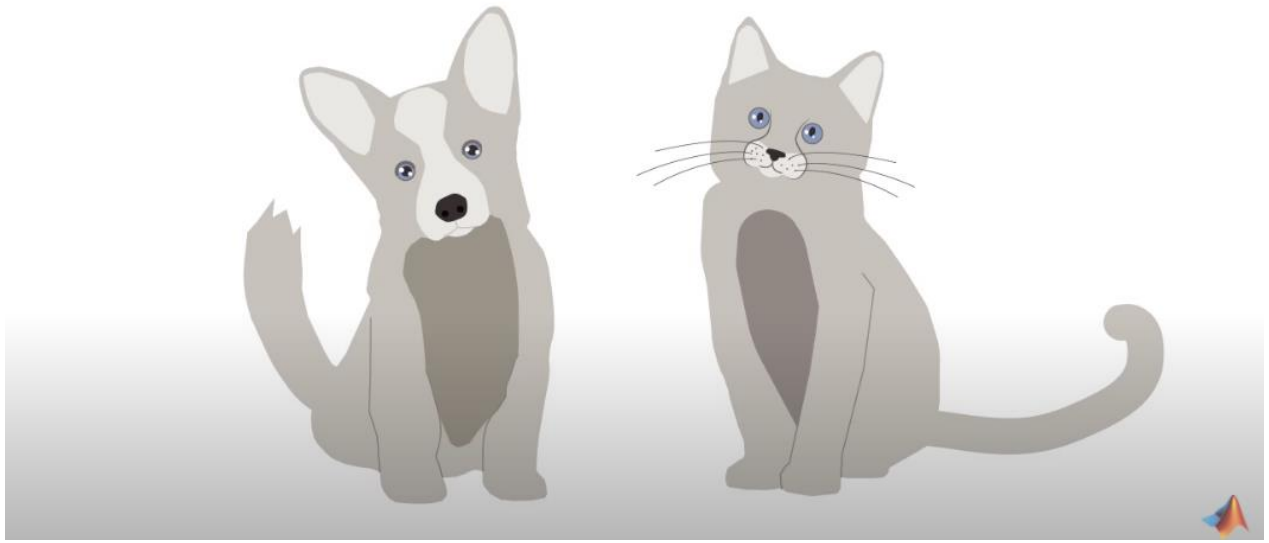
What is Machine Learning



Machine Learning vs. Deep Learning (Video)

Introduction to Deep Learning: Machine Learning vs. Deep Learning

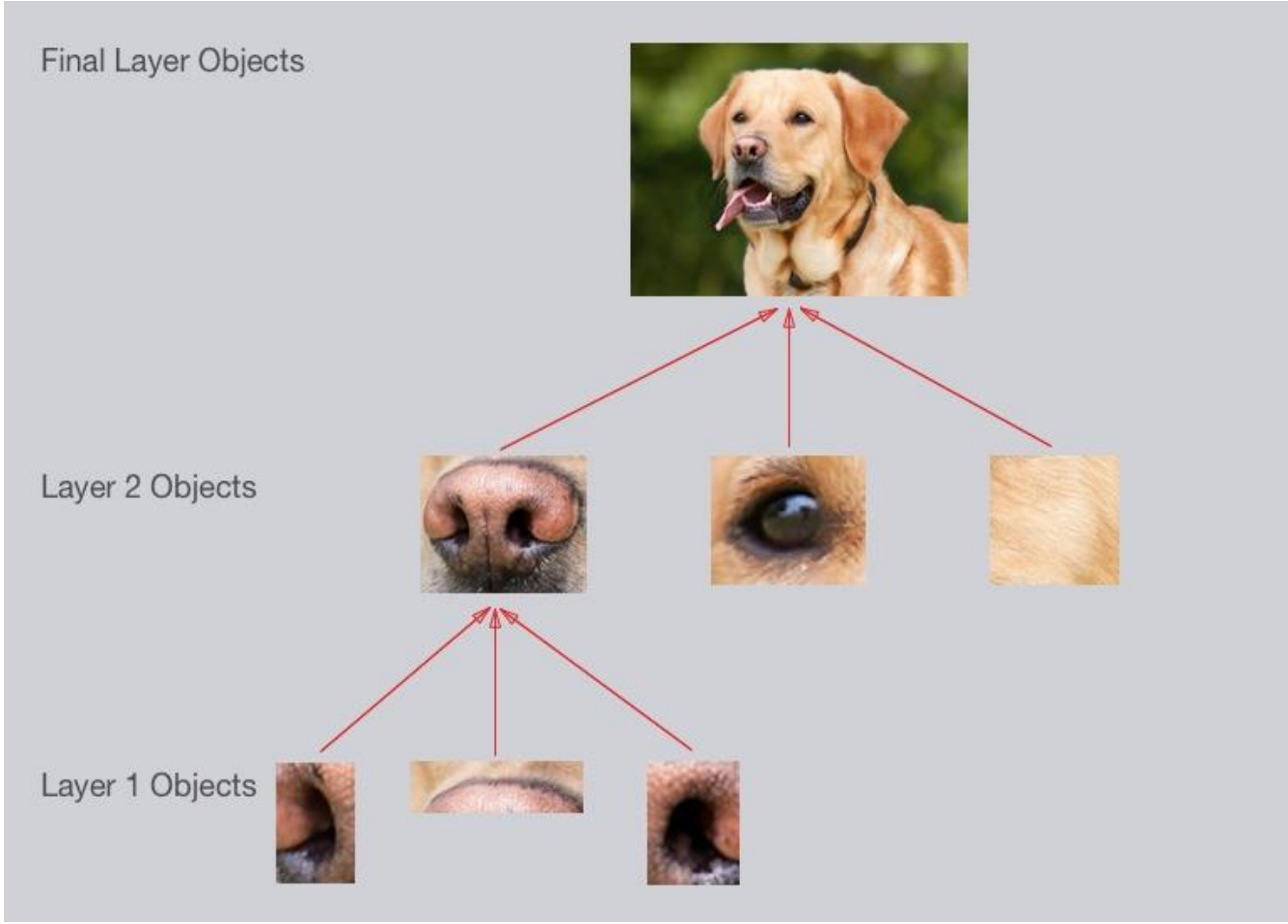




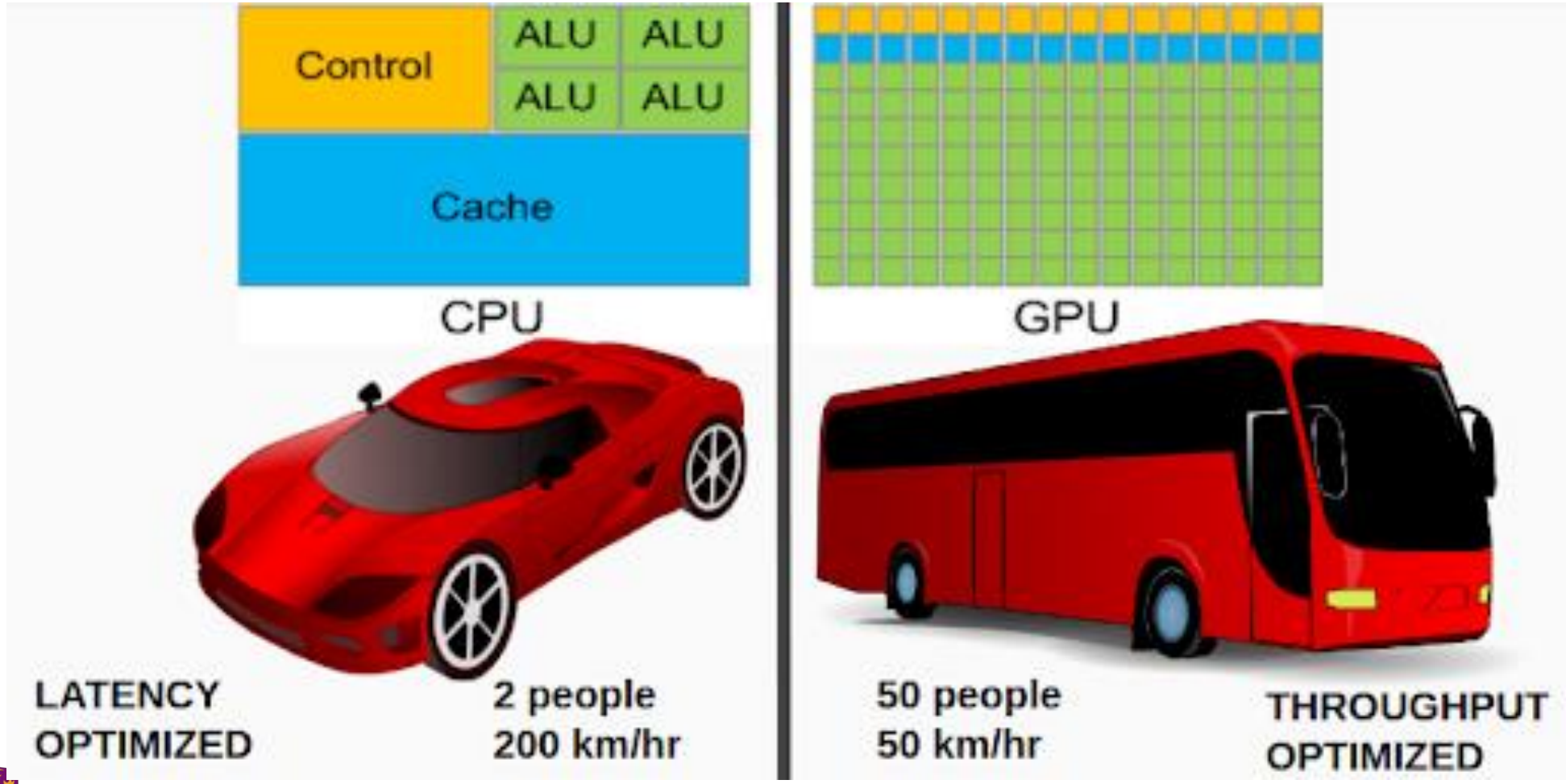
What is Deep Learning

- "Deep" refers to the layered model architectures which are usually deeper than conventional learning models

- Human brain has much deeper layered architecture with increasing levels of abstraction

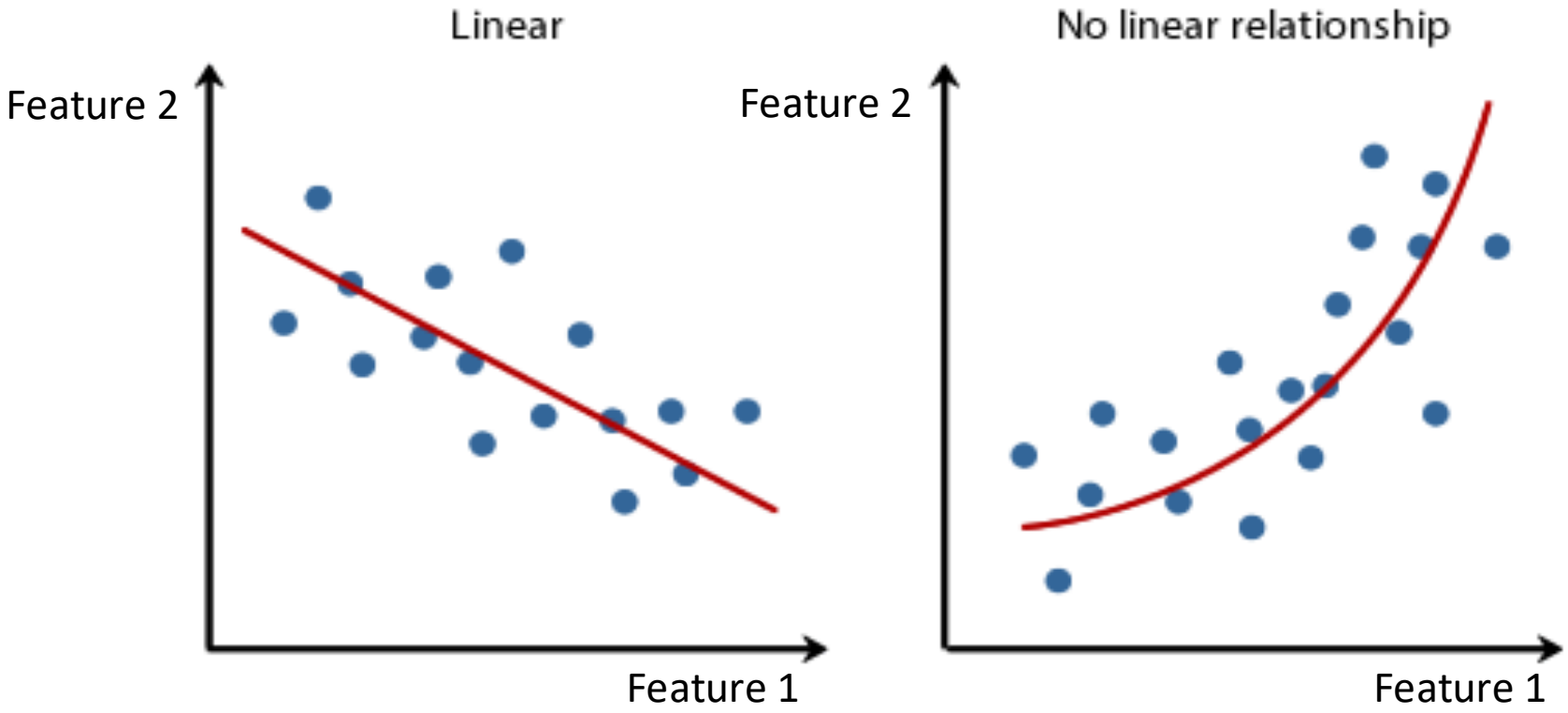


GPU vs. CPU


Control
ALU
ALU
ALU
ALU
Cache
CPU
GPU
LATENCY
OPTIMIZED
2 people
200 km/hr
50 people
50 km/hr
THROUGHPUT
OPTIMIZED

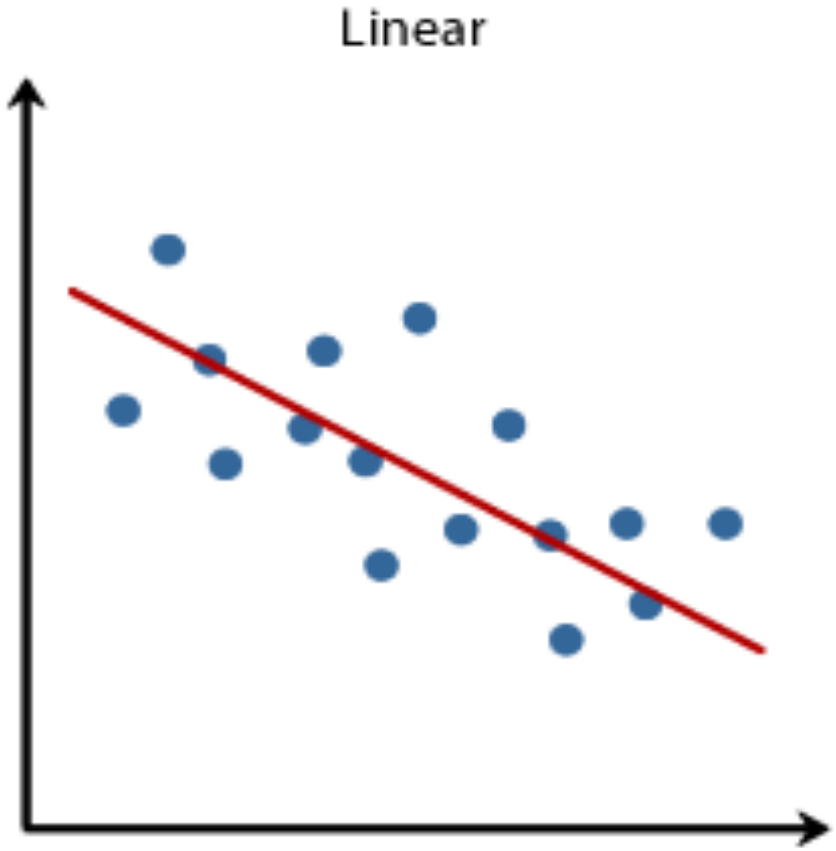
What is Machine Learning - Regression

- Goal: Find an equation that fits the data
- Using a predefined function

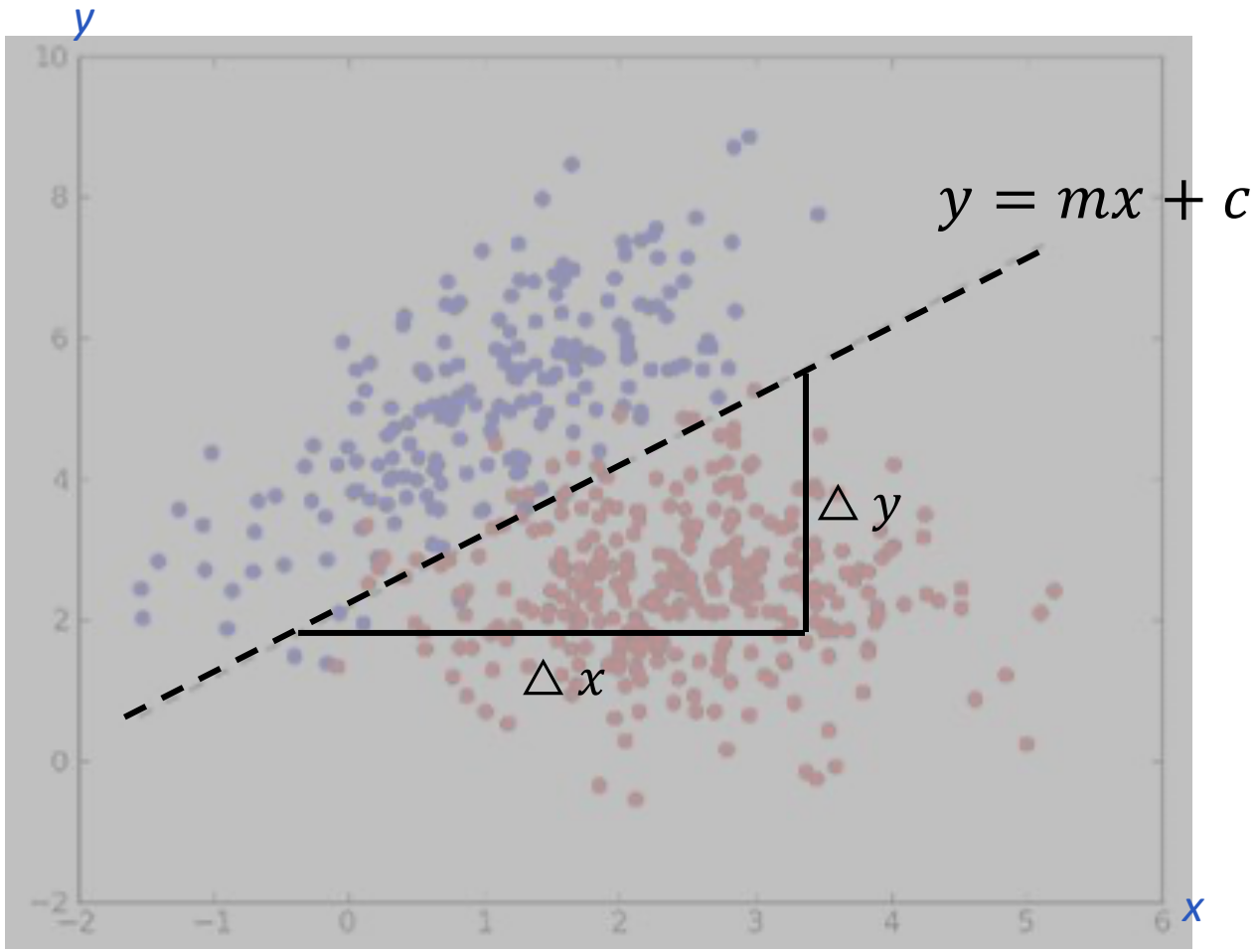


What is Machine Learning

- Data Collections
- Training & Visualization
- Deployment & Fine-tuning



What is the best m and c?





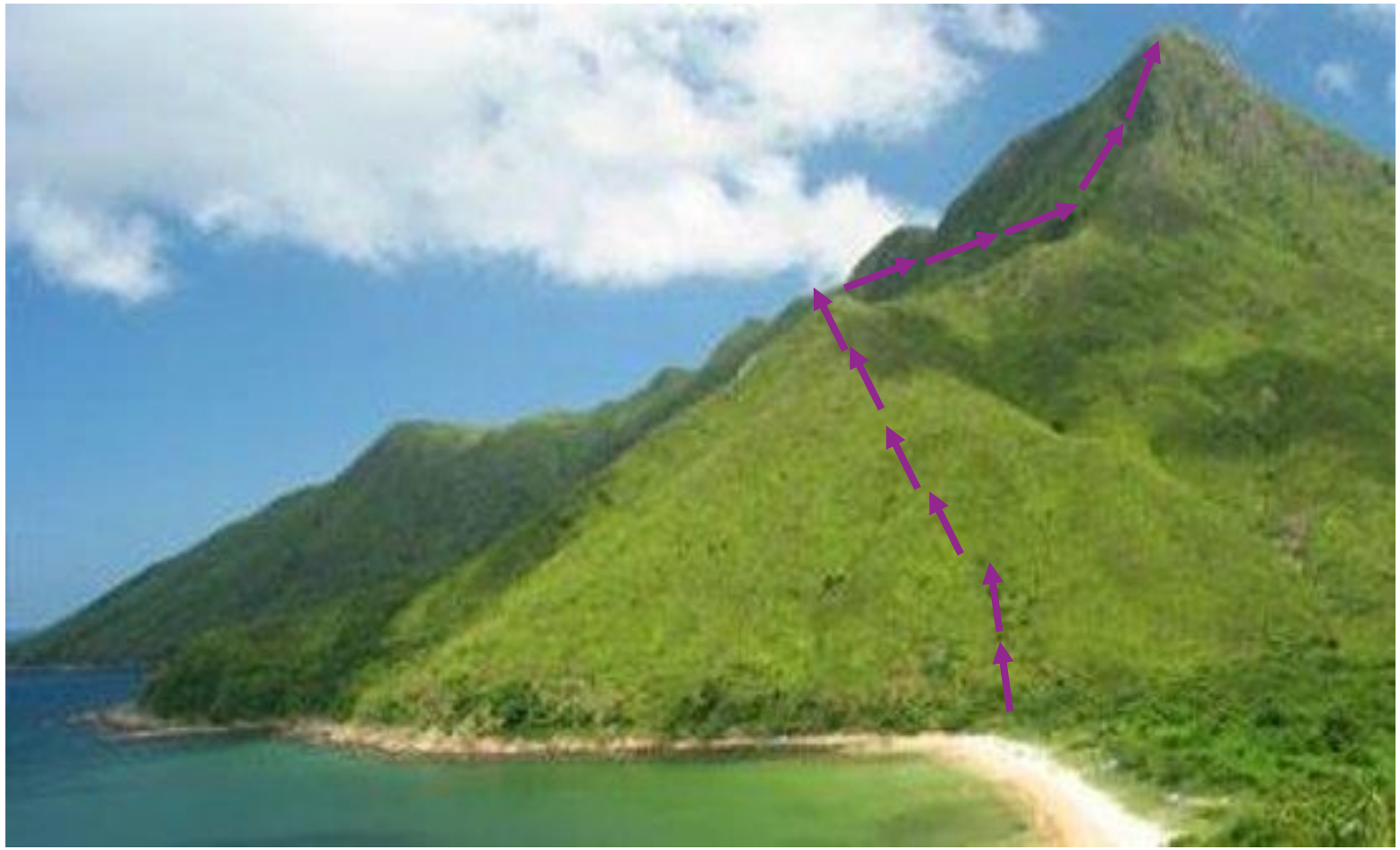
What is Machine Learning - Classification

How to get the best classification result with least error (the peak)?



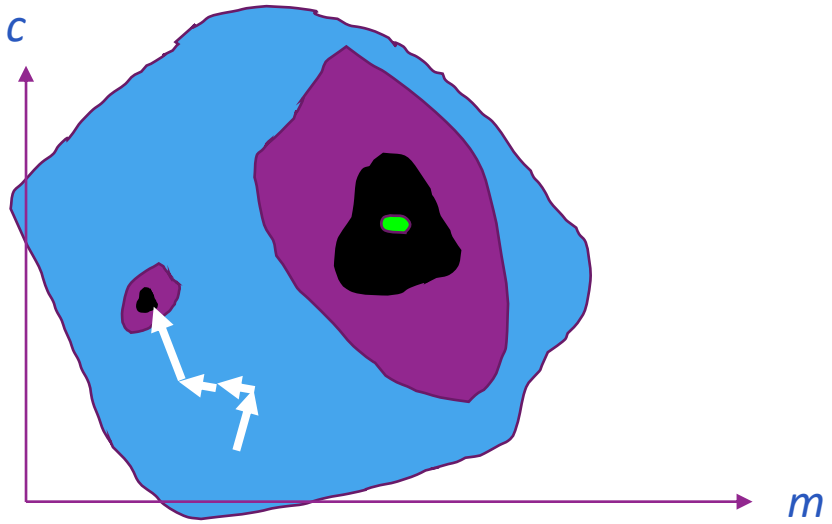


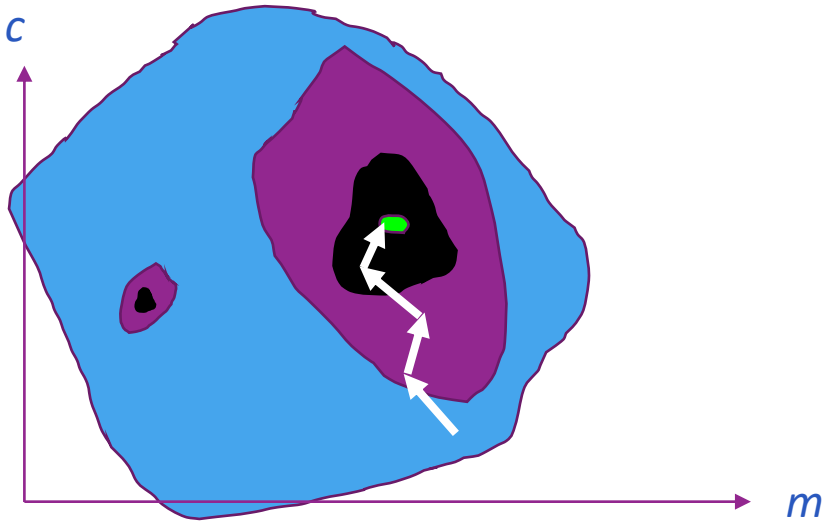
Training – following the slope



What is Machine Learning - Classification

- You start somewhere and you want to reach the peak (least error)
- How's about the data?

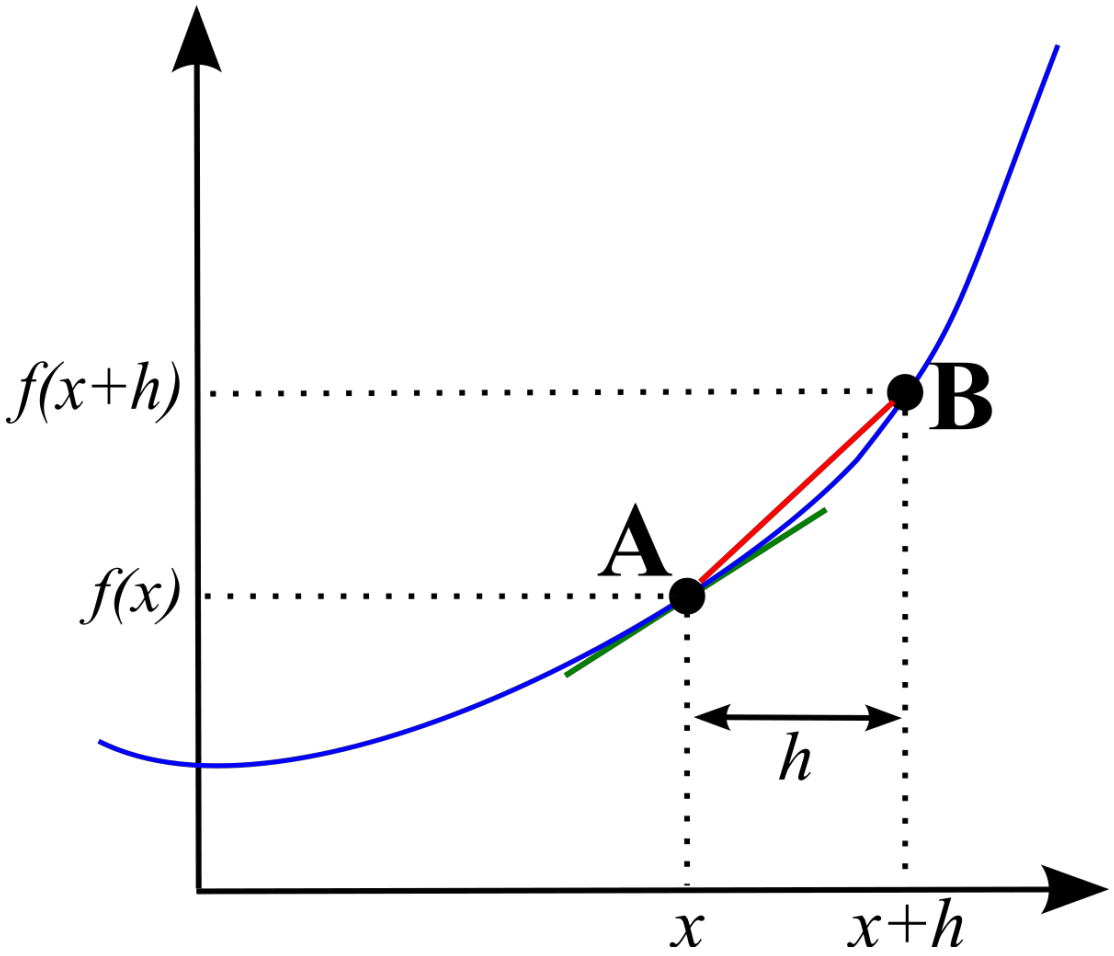




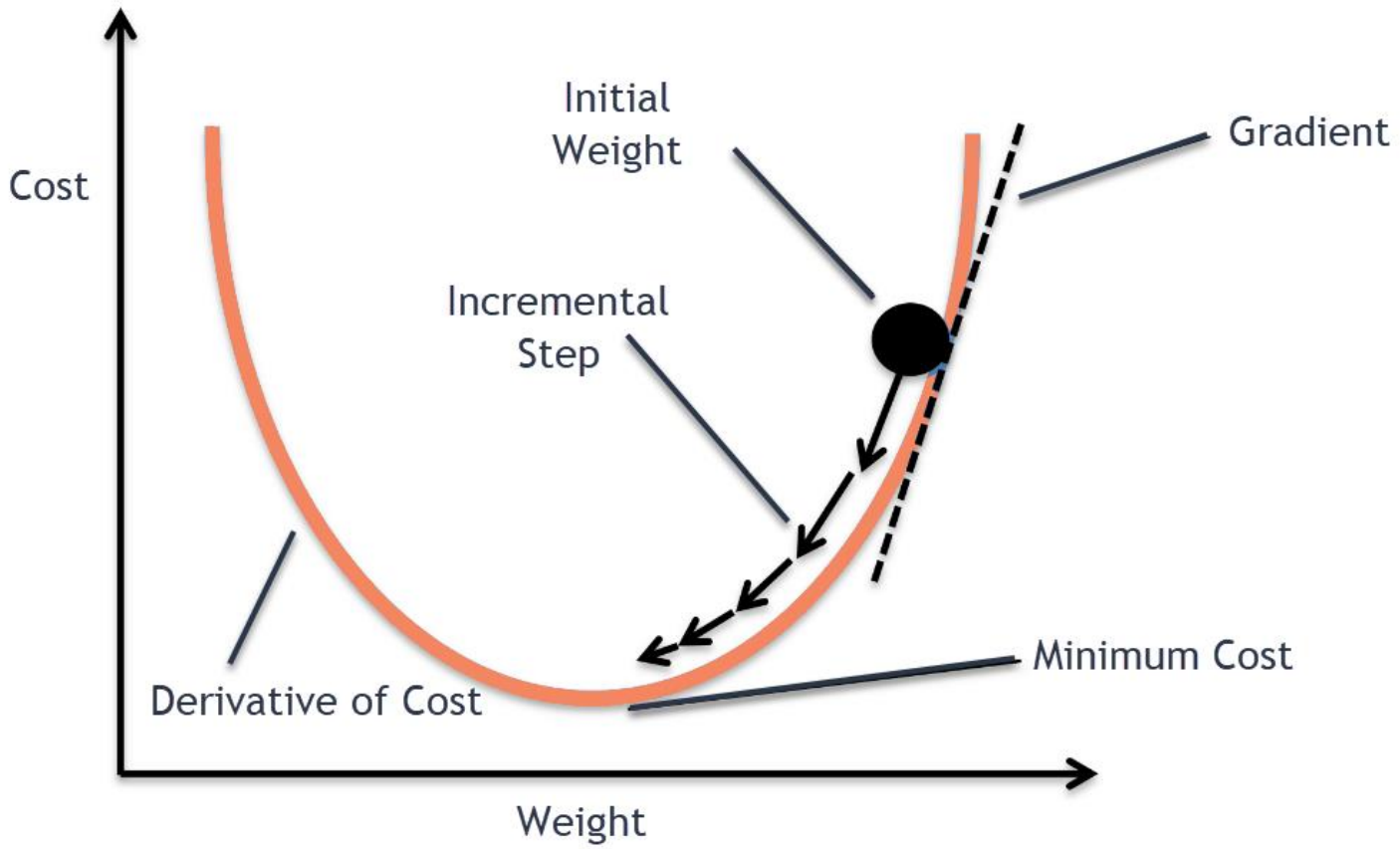


Training – following the slope

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

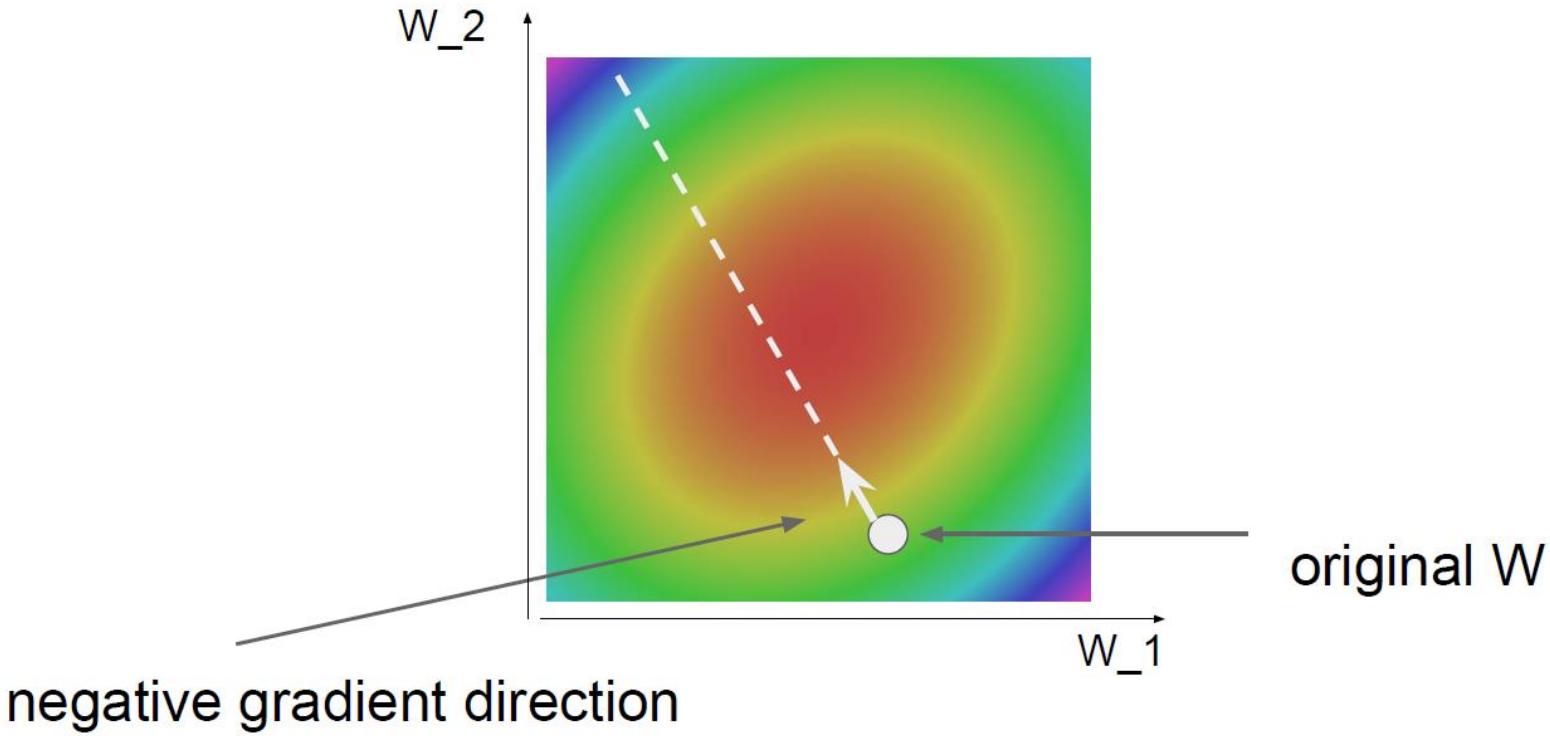


Gradient Descent





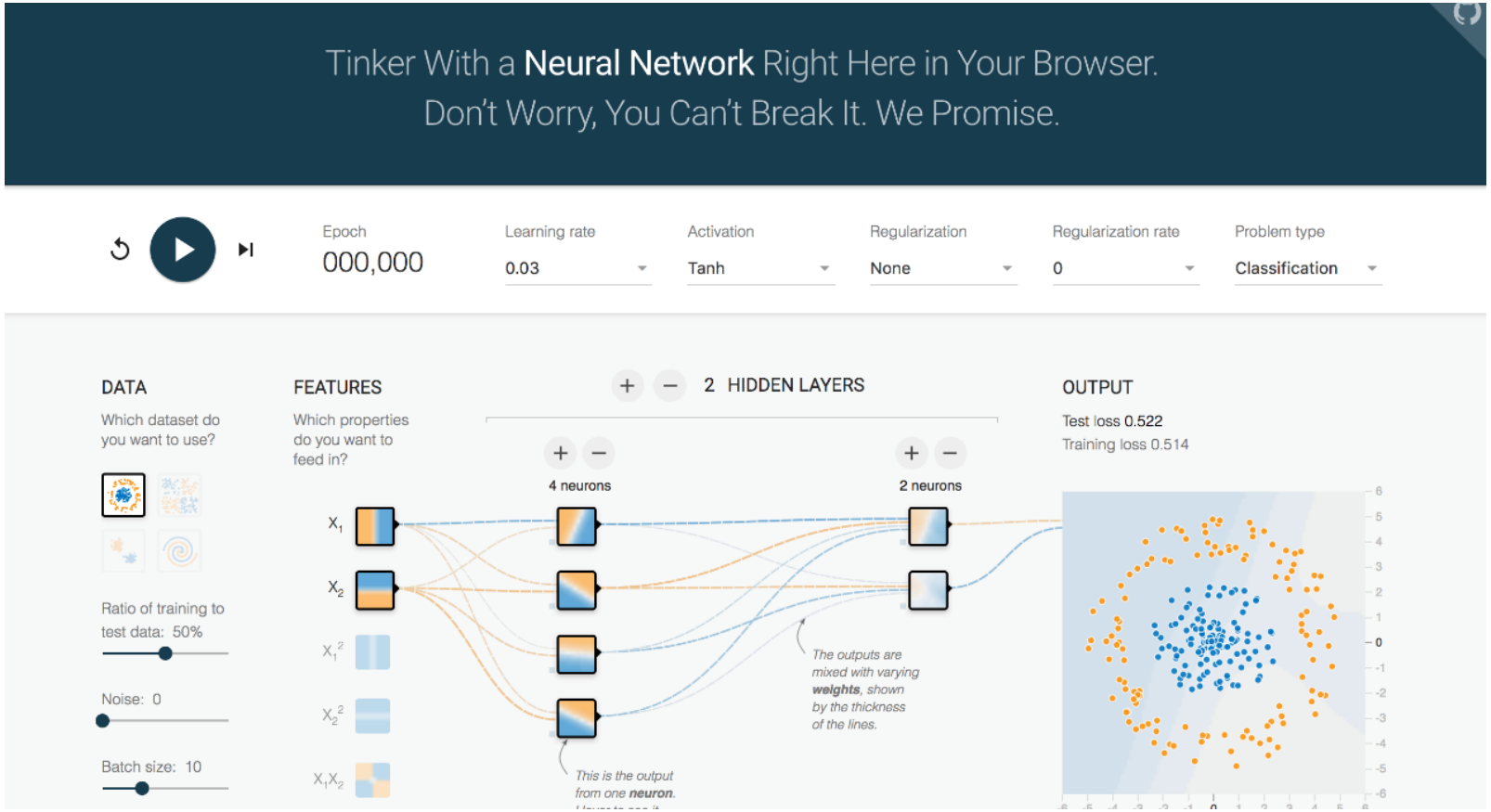
Gradient Descent





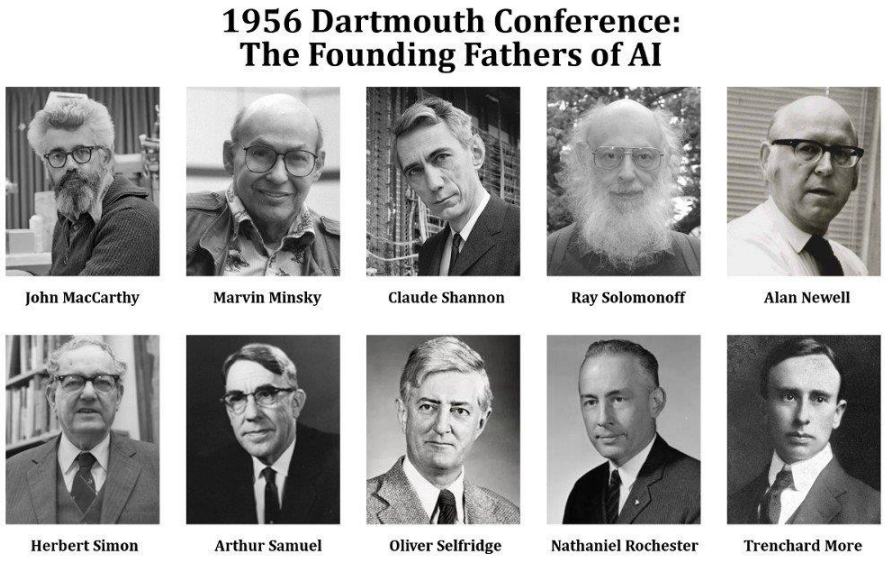
Deep Learning: TensorFlow Playground

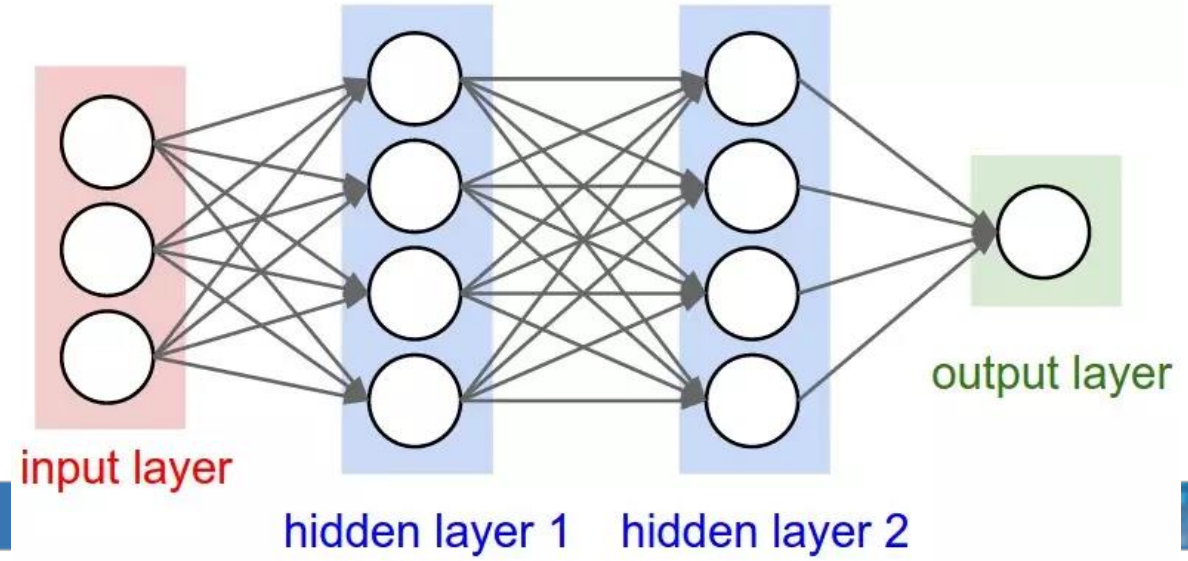
Try deep learning on a browser: http://playground.tensorflow.org/



History of Machine Learning (1950s-1960s)

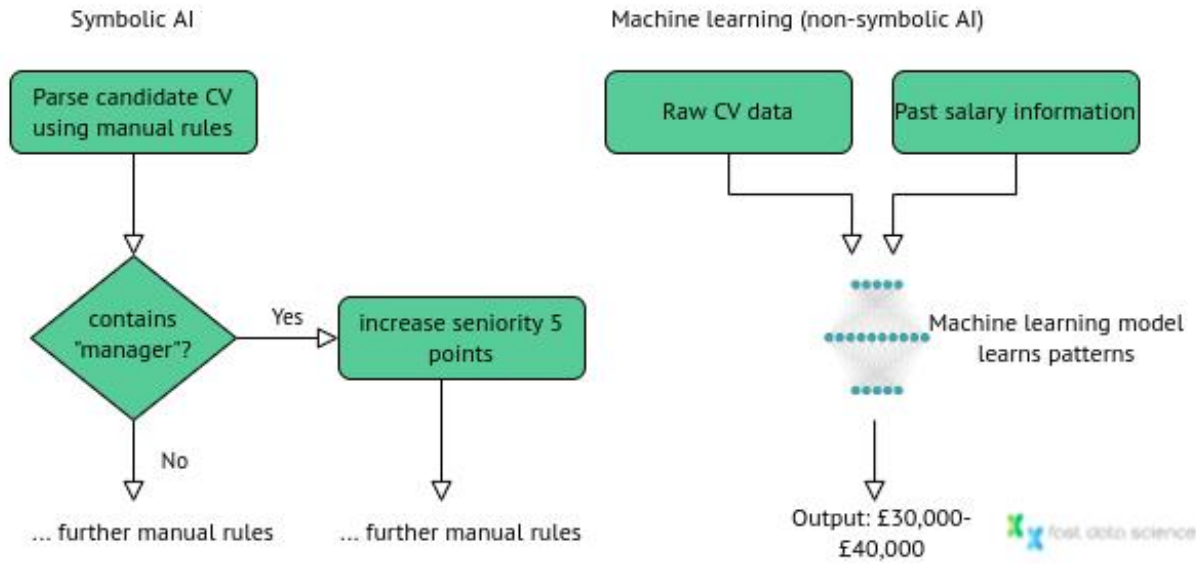
- The Dartmouth Workshop (1956): brought together researchers who aimed to simulate human intelligence using computers.
- Early Neural Networks: developed the concept of perceptrons, which were basic artificial neurons capable of learning. However, perceptrons had limitations, and interest in neural networks declined due to their inability to solve complex problems.





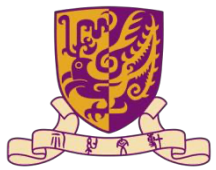
History of Machine Learning (1960s-1980s)

- Symbolic AI and Expert Systems: using explicit rules and logic to solve problems with knowledge bases and reasoning engines
- The Backpropagation Algorithm (1986): The backpropagation algorithm, developed by Geoffrey Hinton, David Rumelhart, and Ronald Williams, allowed for efficient training of multi-layered neural networks



History of Machine Learning (1990s- 2010s)

- Support Vector Machines (SVMs): popular for classify data by finding optimal decision boundaries.
- Rise of Big Data and Data-Driven Approaches: led to increased interest in data-driven approaches, such as decision trees, random forests, and ensemble methods.
- Deep Learning and Neural Networks: multiple layers, achieved remarkable success in various domains, including computer vision, natural language processing, and speech recognition.



History of Machine Learning (2010 – 2020)

- ImageNet Breakthrough (AlexNet): won the ImageNet competition by a wide margin using a Convolutional Neural Network (CNN) trained on GPUs. This event catalyzed the modern deep learning boom and shifted computer vision away from hand-engineered features.
- Generative Adversarial Networks (GANs): a generator network against a discriminator network in a zero-sum game framework, enabling realistic image synthesis and data augmentation.
- The Transformer Architecture: by replacing recurrent structures with multi-head self-attention mechanisms, Transformers enabled massive parallelization during training and became the foundational architecture for modern NLP and vision.



History of Machine Learning (2020+)

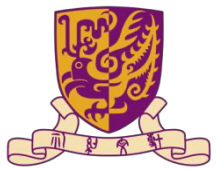
- OpenAI introduced GPT-3 (175 billion parameters), demonstrating that scaling autoregressive Transformer models enables strong few-shot and zero-shot learning capabilities across diverse language tasks without task-specific fine-tuning.
- Denoising Diffusion Probabilistic Models (DDPMs) surpassed GANs in visual generation quality. Systems like Latent Diffusion (Stable Diffusion) made high-fidelity text-to-image synthesis efficient and accessible.
- OpenAI released ChatGPT, pairing Large Language Models with Reinforcement Learning from Human Feedback (RLHF). This unlocked highly coherent instruction-following and conversational interactions, triggering widespread enterprise and consumer adoption.



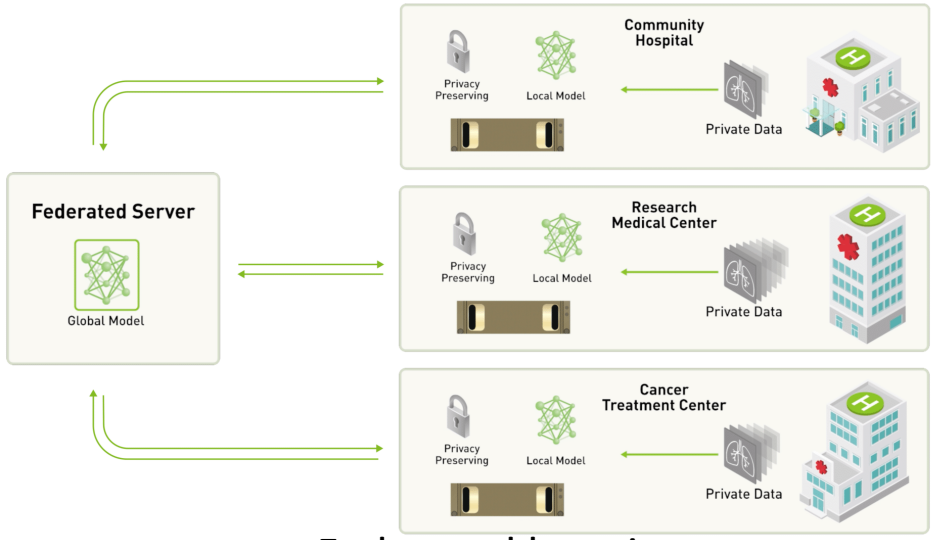
History of Machine Learning (2020+)

- Machine learning shifted toward natively multimodal architectures capable of processing text, vision, audio, and code simultaneously. Advances in reasoning-focused architectures, Mixture of Experts (MoE), parameter-efficient fine-tuning (PEFT)

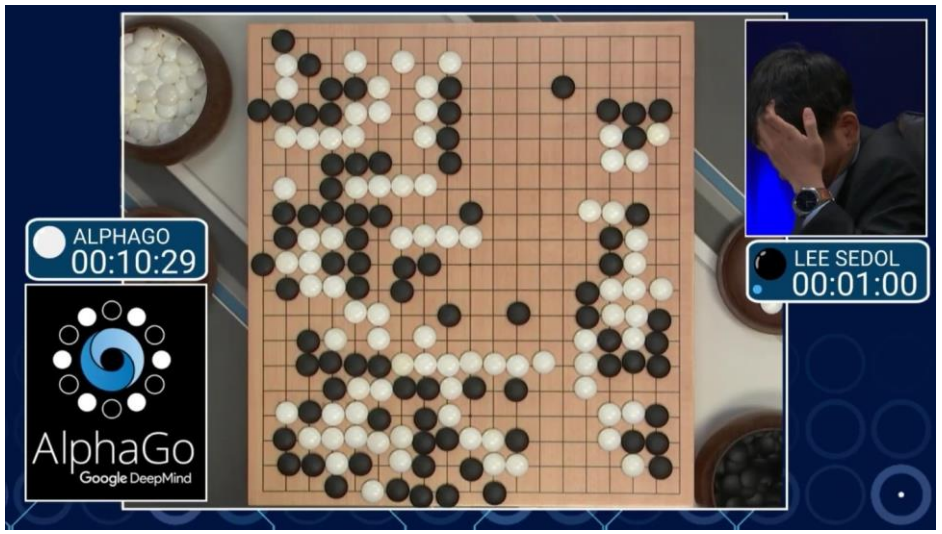
- Agentic Workflow: Decomposes complex tasks via chain-of-thought, tree-of-thought, or self-reflection loops to correct errors dynamically. Implemented the capacity to execute external tools including code interpreters, web search, API endpoints, databases, and filesystem operations.



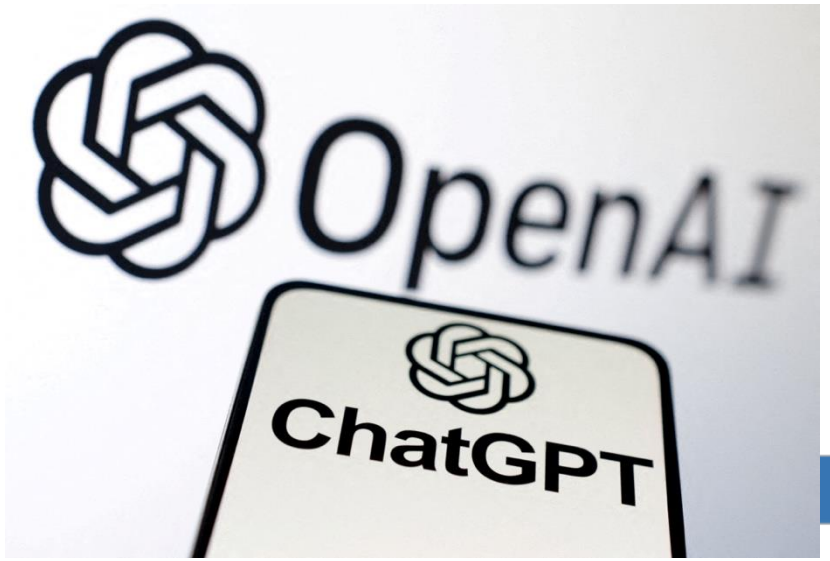
History of Machine Learning



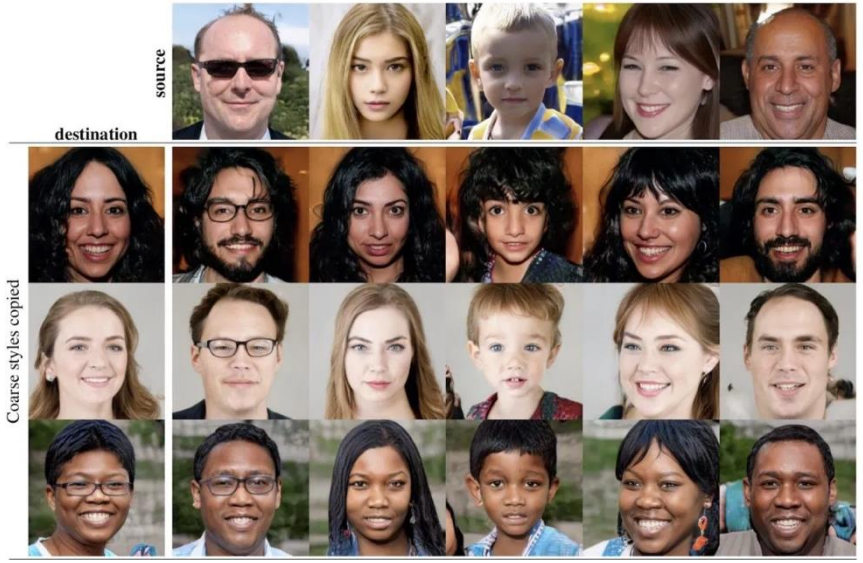
Federated learning



Reinforcement Learning

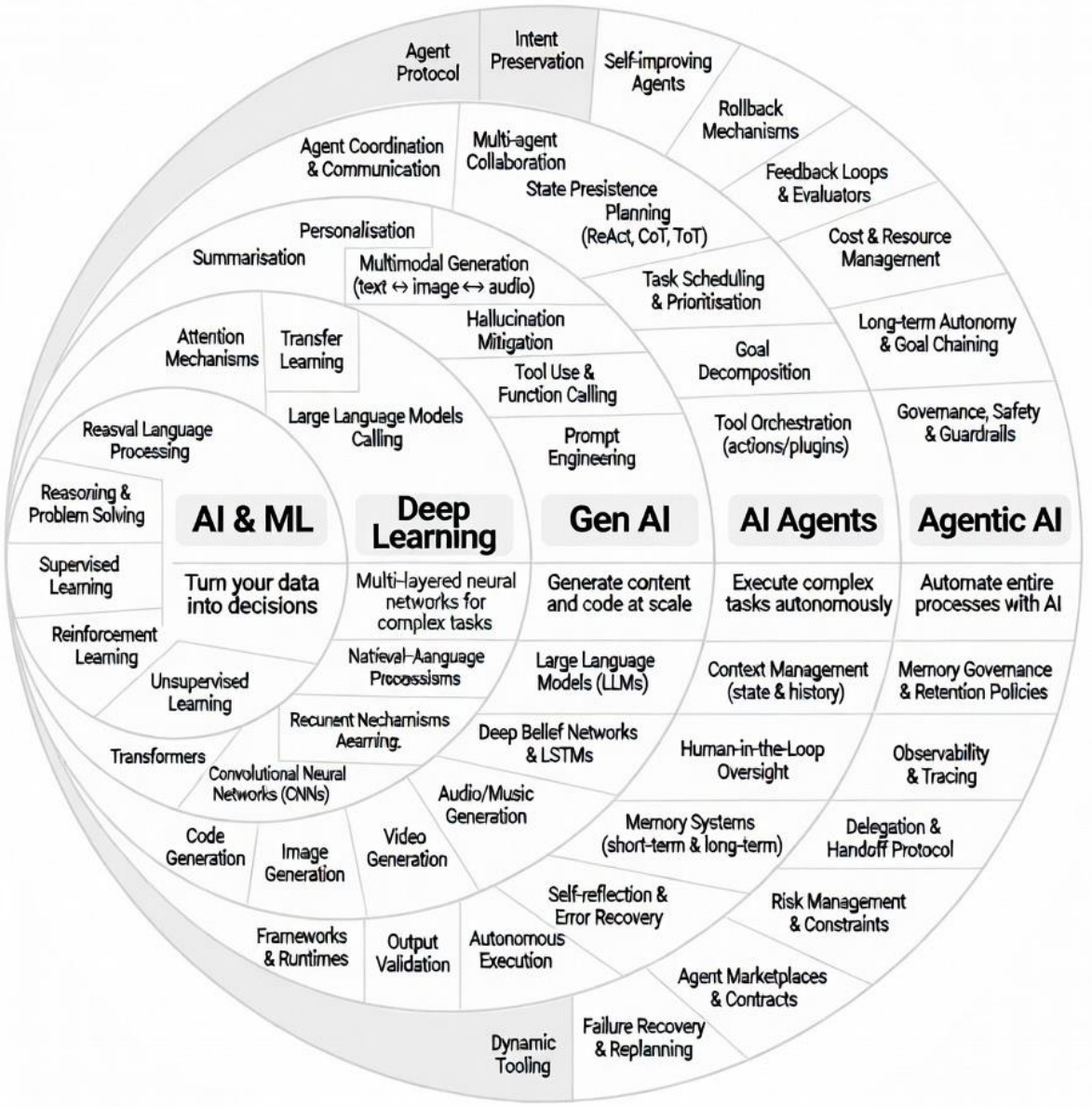


Large language model



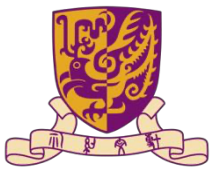
Generative AI

Agentic AI


AI & ML
Turn your data into decisions
Deep Learning
Multi-layered neural networks for complex tasks
Gen AI
Generate content and code at scale
AI Agents
Execute complex tasks autonomously
Agentic AI
Automate entire processes with AI
Reasoning & Problem Solving
Supervised Learning
Reinforcement Learning
Unsupervised Learning
Reasval Language Processing
Transformers
Attention Mechanisms
Transfer Learning
Large Language Models Calling
Convolutional Neural Networks (CNNs)
Recurrent Mechanisms Aearning.
Natieval-Aanguage Processisms
Large Language Models (LLMs)
Deep Bellef Networks & LSTMs
Prompt Engineering
Tool Use & Function Calling
Hallucination Mitigation
Multimodal Generation (text ↔ image ↔ audio)
Personalisation
Summarisation
Audio/Music Generation
Video Generation
Image Generation
Code Generation
Frameworks & Runtimes
Output Validation
Autonomous Execution
Dynamic Tooling
Failure Recovery & Replanning
Self-reflection & Error Recovery
Planning (ReAct, CoT, ToT)
Multi-agent Collaboration
Agent Coordination & Communication
Agent Protocol
Intent Preservation
Self-improving Agents
Rollback Mechanisms
Feedback Loops & Evaluators
Cost & Resource Management
Long-term Autonomy & Goal Chaining
Governance, Safety & Guardrails
State Presistence
Task Scheduling & Prioritisation
Goal Decomposition
Tool Orchestration (actions/plugins)
Context Management (state & history)
Memory Systems (short-term & long-term)
Human-in-the-Loop Oversight
Observability & Tracing
Memory Governance & Retention Policies
Delegation & Handoff Protocol
Risk Management & Constraints
Agent Marketplaces & Contracts

Agenda

- Course Introduction
- What are machine learning
- Supervised Learning and KNN



Learning a Class from Examples

- Goal: Identify family cars from all cars. Class 1- family cars; Class 0- others
- Feature engineering: find features shared by all class 1 but not class 0.
- Prediction: is the car x (not in training data) a family car?
- Represent each car x as a feature vector $x = (x_1, x_2, \ldots, x_n)^T$
  - e.g., x_1 = price, x_2 = engine power.



Training Set S_{train}

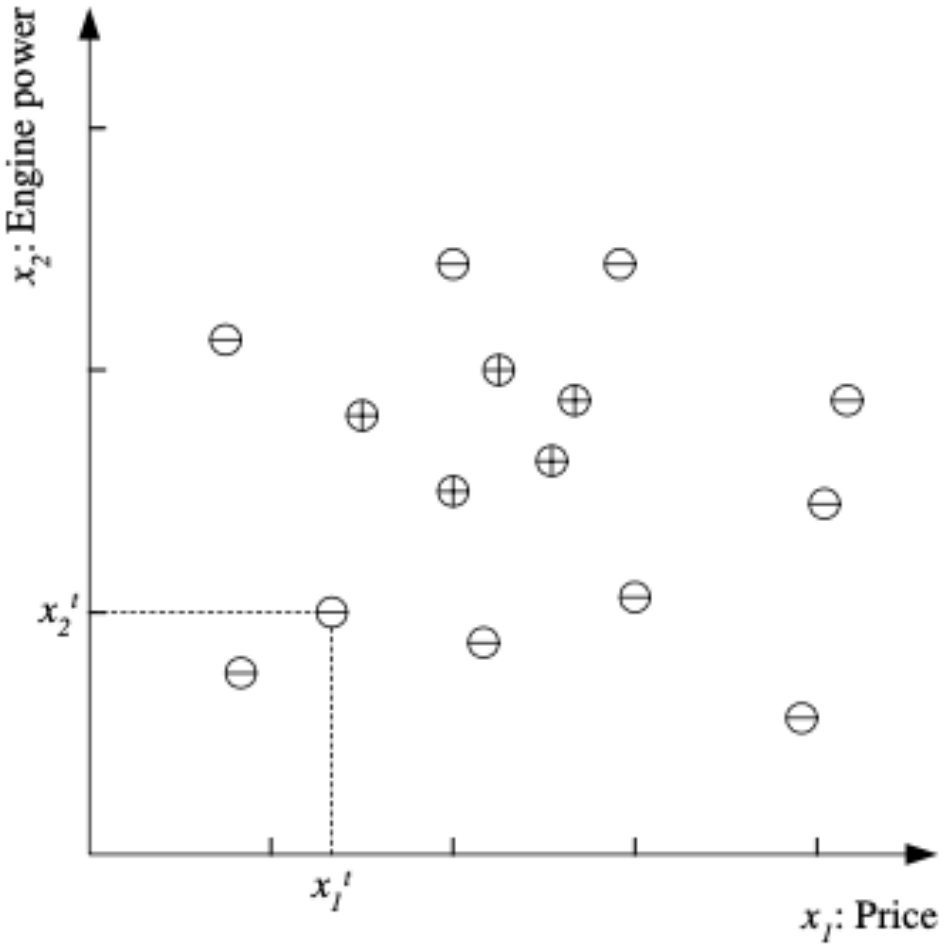
- A set of n examples:

$$S_{train} = (x_1, y_1), \ldots, (x_n, y_n)$$

- Each (x_i, y_i) is a car
- Feature vector (2 features):

$$x_i = (f_{i1}, f_{i2})$$

- Class label:
  - $y = \begin{cases} 1 & \text{for family } cars\ (+) \\ 0 & \text{for others } (-) \end{cases}$



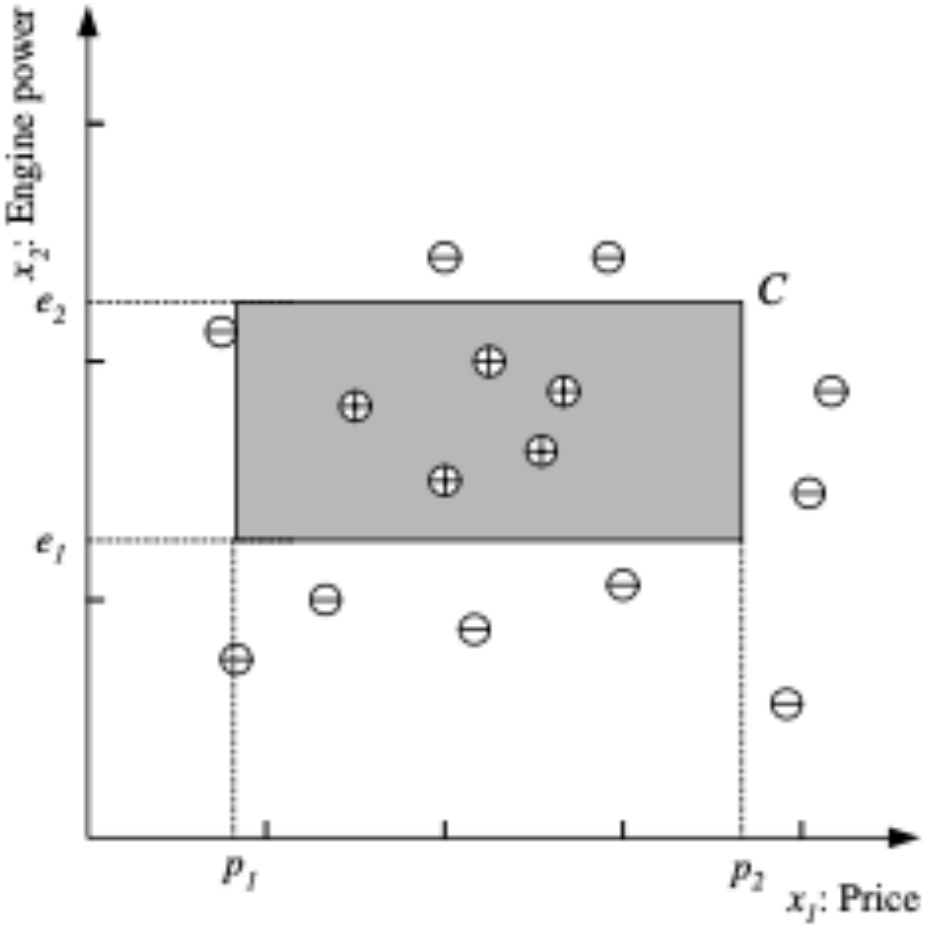


Hypothesis Class C

- The rectangle corresponds to a model (a.k.a. hypothesis) specified by 4 points: $(p_1, e_1), (p_2, e_1), (p_1, e_2)$ and (p_2, e_2)

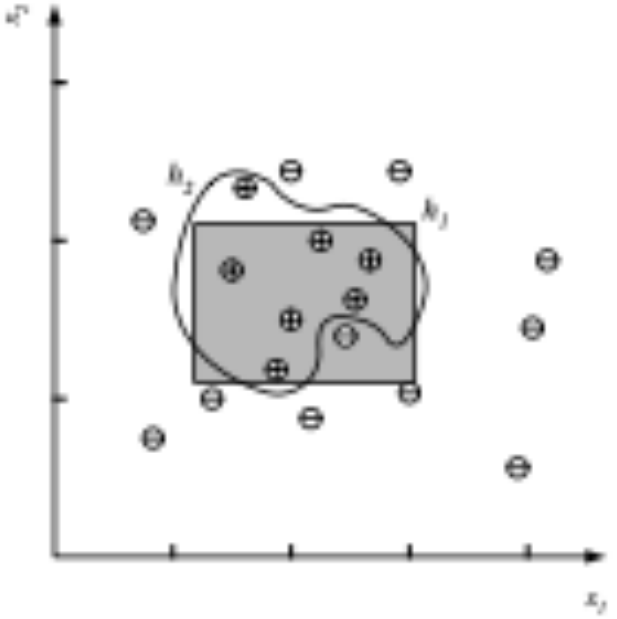
- The learning algorithm finds a hypothesis h to approximate the class C as closely as possible.

- Make sure that the hypothesis class H is flexible enough (or has enough capacity) to learn C , i.e., to find h that is as similar as possible to C.



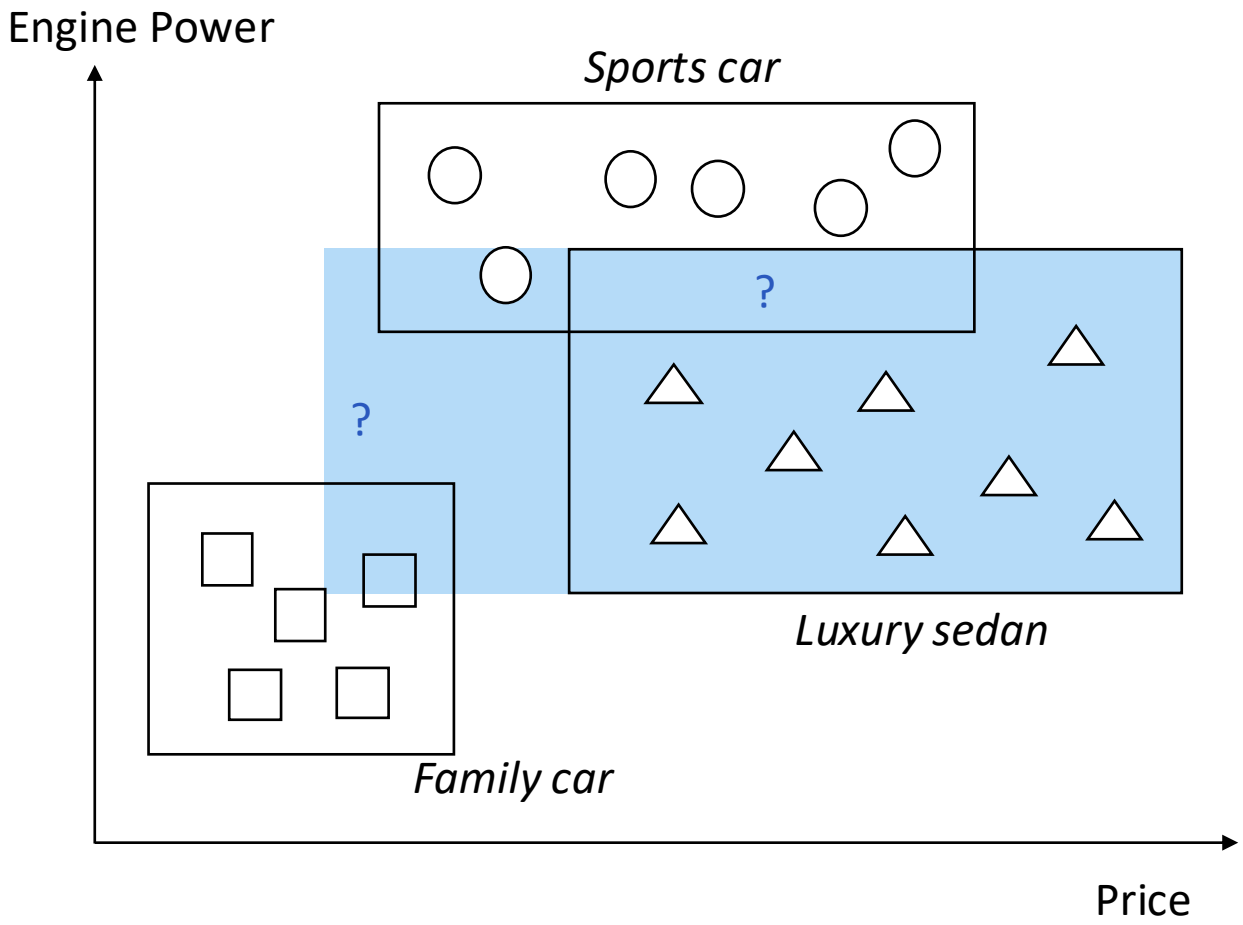
Noise and Model Complexity

- Sources of noise:
  - Measurement (input)
  - Labelling (output)
  - Hidden attributes



- Simple rectangle (linear) model is preferred
  - Simpler to use with lower computational complexity
  - Easier to train with lower space complexity
  - Easier to explain with higher interpretability
  - Better generalization with lower variance (Occam's razor)




Engine Power
Sports car
?
?
Luxury sedan
Family car
Price

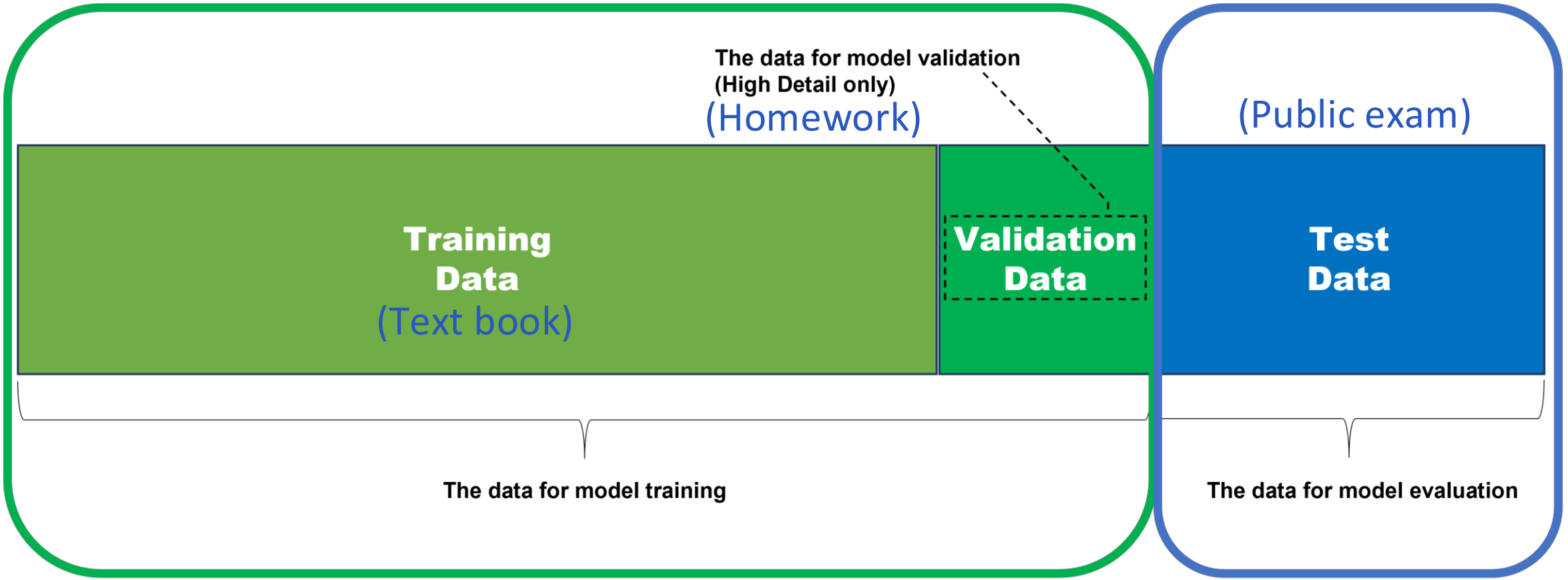


Process of Training

- Build and train a model with only Training data, and measure the performance with the validation data
- For different hyperparameters / models, measure the performance with the validation data
- With the best hyperparameters / models, measure using testing data (in this course, it is the submission)

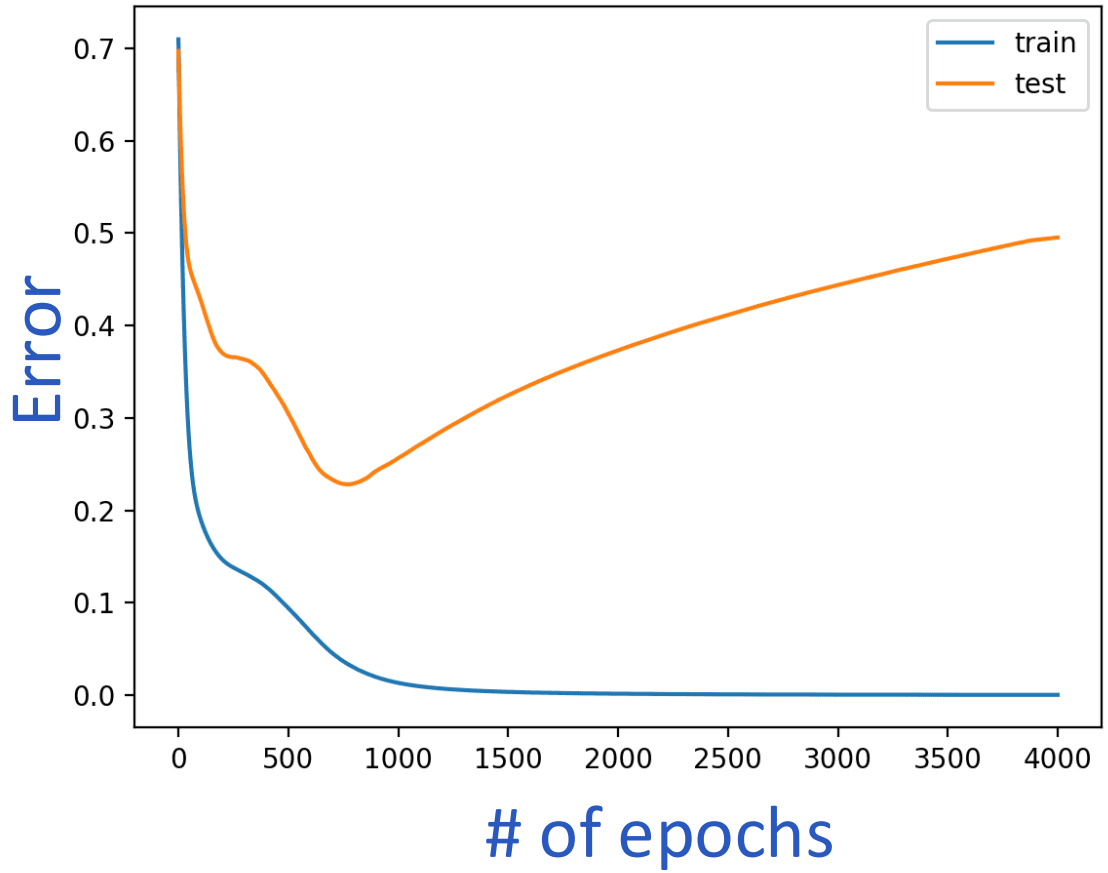


Training, Testing and Validation Sets



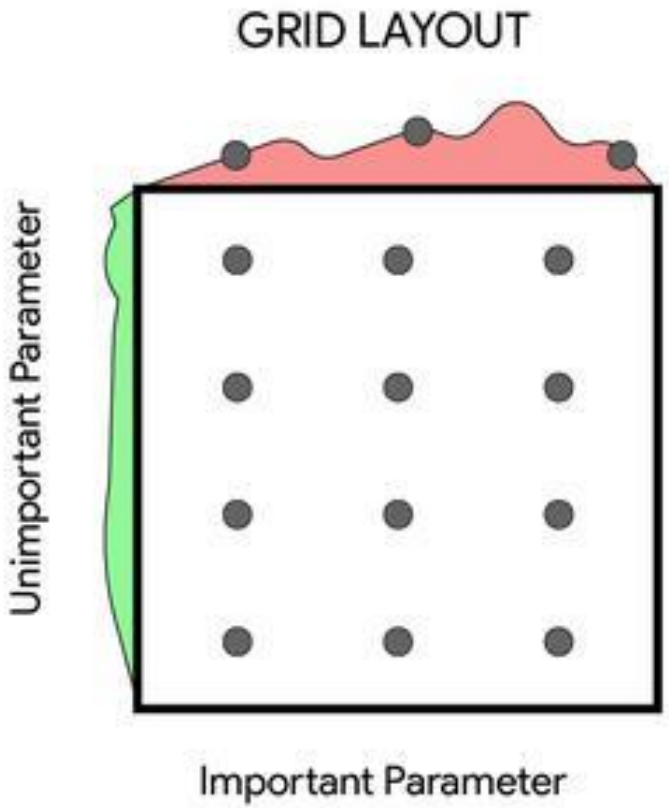
Process of Training

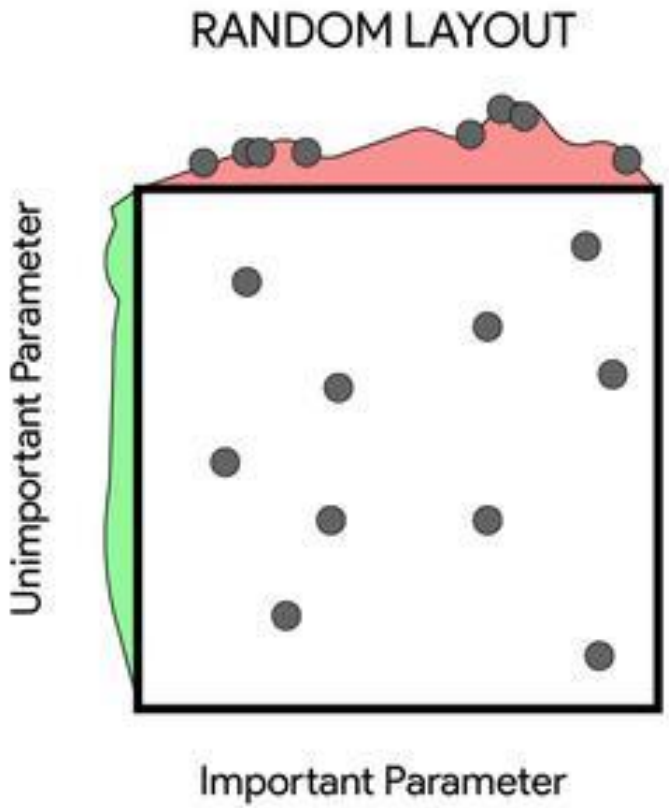
- Training set: drops at the beginning, saturated after
- Testing set: drops at the beginning but higher than training set, and goes up after a point (overfitted)





Tuning models using Grid Search







Tuning models using Grid Search

```
# Hyperparameter grid
param_grid = {
    'hidden_size': [32, 64, 128],
    'learning_rate': [0.001, 0.01],
    'num_epochs': [50, 100]
}

# Convert param_grid to a list of parameter combinations
param_combinations = list(ParameterGrid(param_grid))

# Function to train the model
def train_model(hidden_size, learning_rate, num_epochs):
    input_size = X.shape[1]  # Number of features
    output_size = y.shape[1]  # Output dimension

    model = FeedforwardNN(input_size, hidden_size, output_size)
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_

    for epoch in range(num_epochs):
        for batch_X, batch_y in train_loader:
            optimizer.zero_grad()  # Clear gradients
            outputs = model(batch_X)  # Forward pass
            loss = criterion(outputs, batch_y)  # Compute loss
            loss.backward()  # Backward pass
            optimizer.step()  # Update weights

    return model
```

```
# Grid Search
best_model = None
best_loss = float('inf')
best_params = {}

for params in param_combinations:
    print(f"Training with parameters: {params}")
    model = train_model(params['hidden_size'], params['learning_rate'], params['num_e
pochs'])

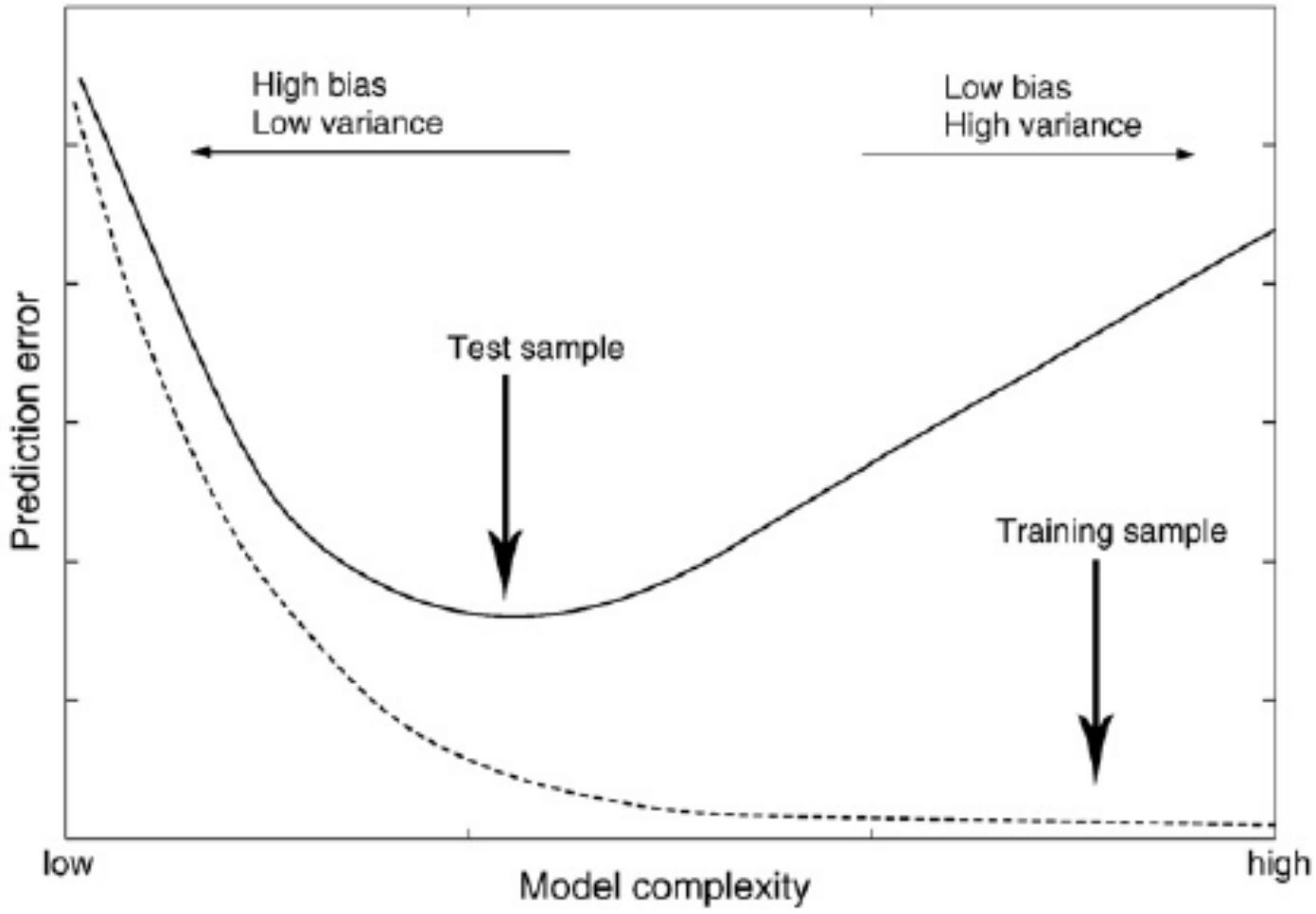
    # Evaluate model (using validation set or a simple train loss)
    with torch.no_grad():
        total_loss = 0
        for batch_X, batch_y in train_loader:
            outputs = model(batch_X)
            loss = nn.MSELoss()(outputs, batch_y)
            total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)
    print(f"Average Loss: {avg_loss:.4f}")

    # Check if this model is the best one
    if avg_loss < best_loss:
        best_loss = avg_loss
        best_model = model
        best_params = params

print(f"Best parameters: {best_params}, Best loss: {best_loss:.4f}")
```

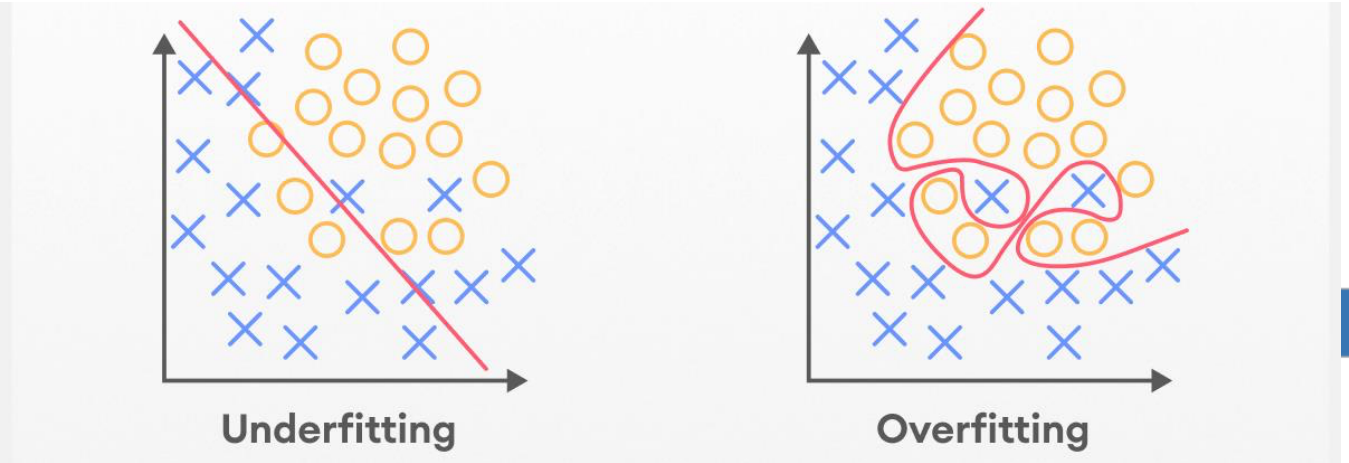
Model Complexity

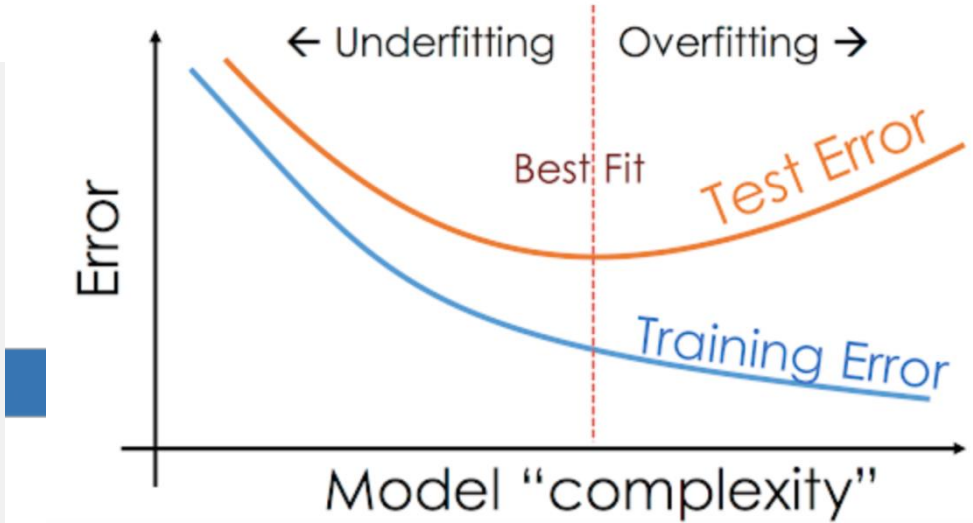




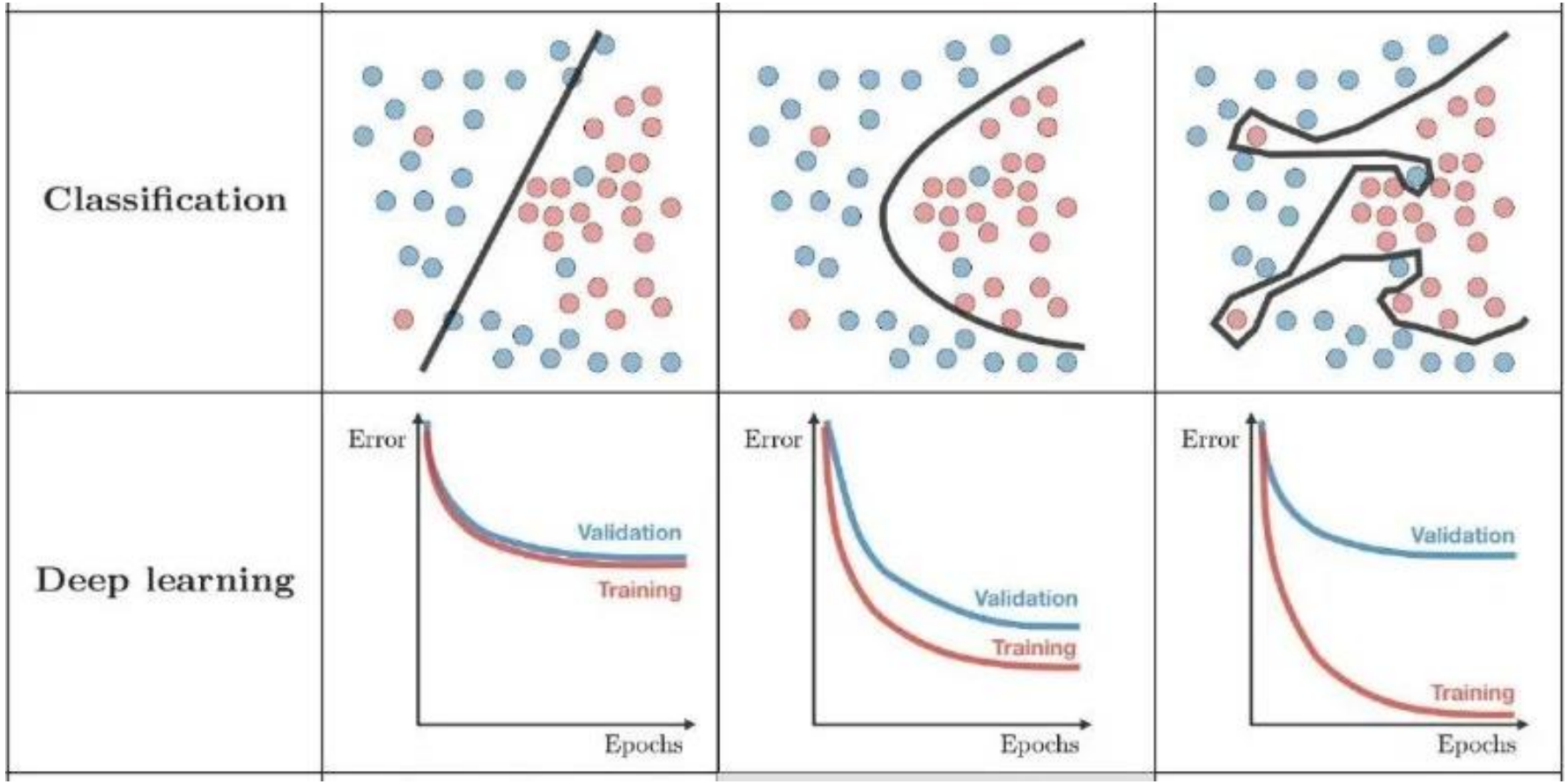
Model selection

- Learning is an ill-posed problem
  - Data alone is not sufficient to find a unique solution
- Inductive bias (i.e., assumptions about H) is needed.
- Generalization: how well a learned model performs on new data.
  - Overfitting: H is more complex than C
  - Underfitting: H is less complex than C
- Model selection: choosing the right inductive bias.





Model selection



How do we process the Big Data?

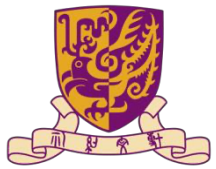
Python!



Python

- Easy to use and easy to maintain
- Portable library with supports from GUI programing
- Cross platform: Mac, Windows and Unix
- Free and open-source





Source: https://www.slideshare.net/EdurekaIN/python-ppt

NumPy

- Library for the Python programming language
- Support for large, multi-dimensional arrays and matrices
- Useful for linear algebra, Fourier transform, and random number capabilities



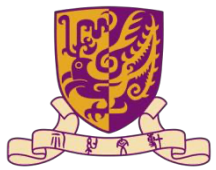
NumPy Examples

Need to import NumPy (and other libraries)

```
import numpy as np #initial a library

a = np.mat([1,2,3]) #initial a matrix

a.shape #get the shape of the matrxi
```



NumPy Indexing

The index start from “0”, negative indexing are also supported

```
x = np.arange(10) #initial a matrix with: [0,1,2,3,…,9]

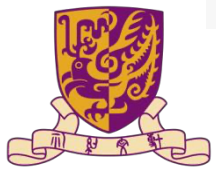
x[2] #the result is 2

x[-2] #the result is 8

x.shape = (2,5) # now x is 2-dimensional

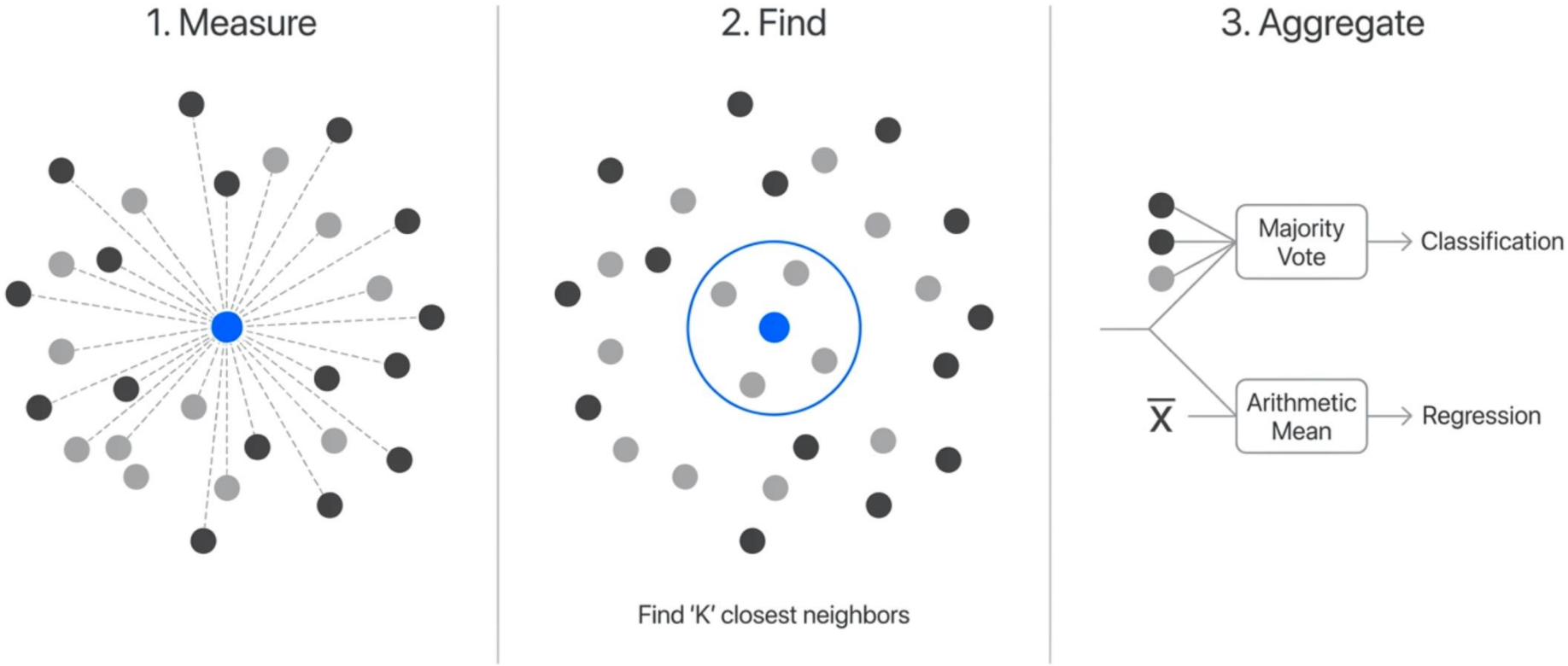
x[1,3] #the result is 8

x[1,-1] #the result is 9
```



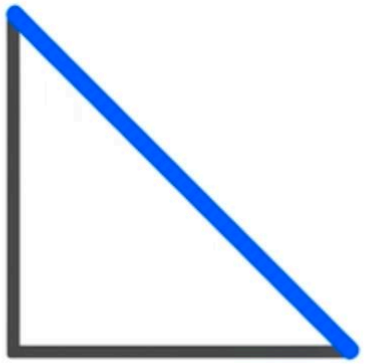
K-Nearest Neighbors

- KNN is a lazy learner. It builds no mathematical model during training; it simply memorizes the dataset and defers all computation to the moment a query is made.



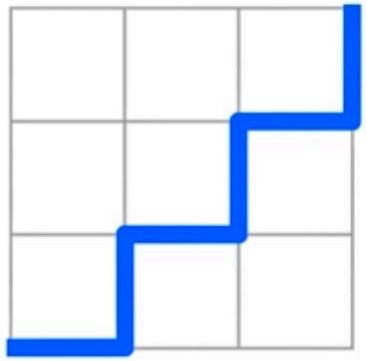
The Three Foundations of Distance

Euclidean (L2 Norm)



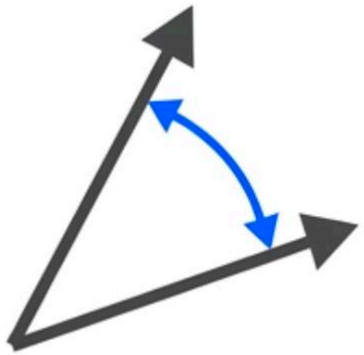
- Straight-line geometric distance.
- Best for continuous, low-dimensional, isotropic feature spaces.

Manhattan (L1 Norm)



- Sum of absolute coordinate differences.
- Best for high-dimensional, sparse spaces; highly robust to outliers.

Cosine Similarity

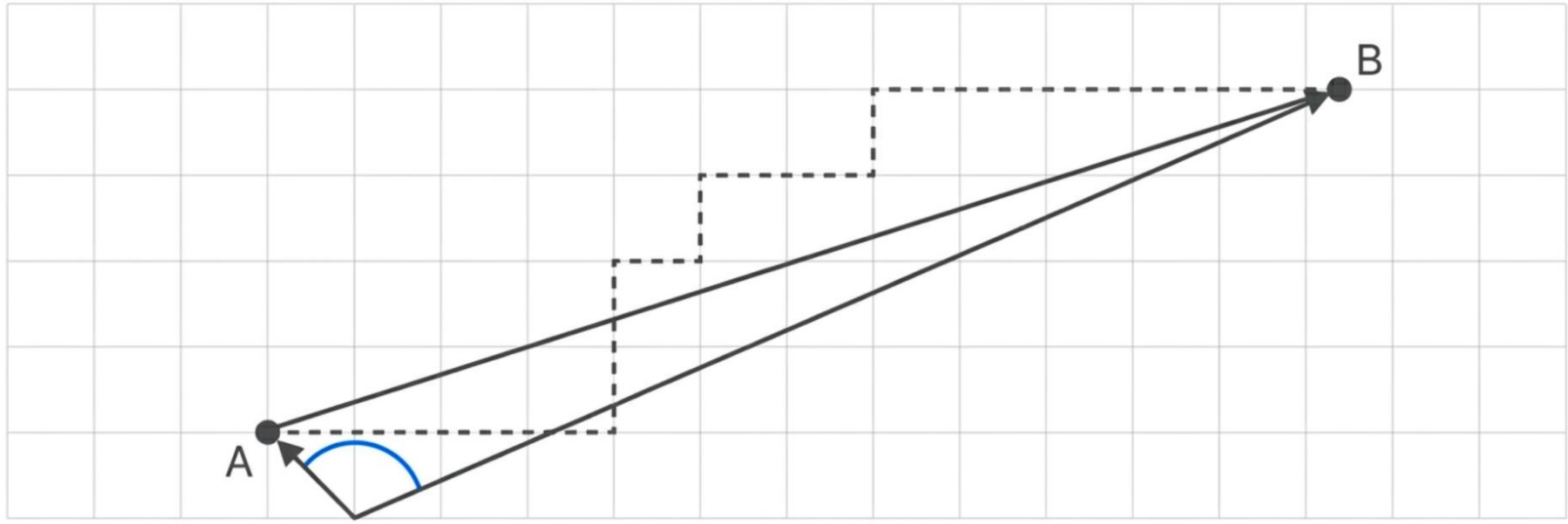


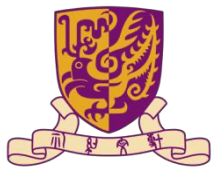
- Measures directional alignment, ignoring magnitude.
- Best for scale-independent text processing and document representation.



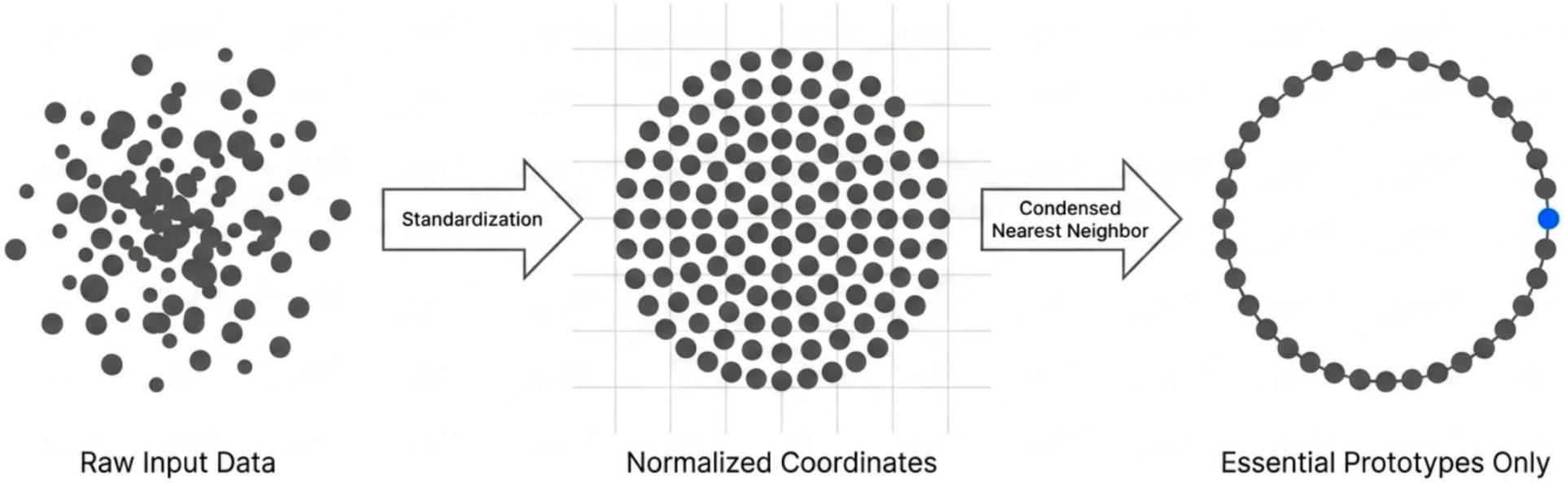
First, you need a space

- A KNN algorithm is dependent on how closeness is defined
- Changing the mathematical rules for measuring space fundamentally alters which neighbors are considered closest





Preparing the Space

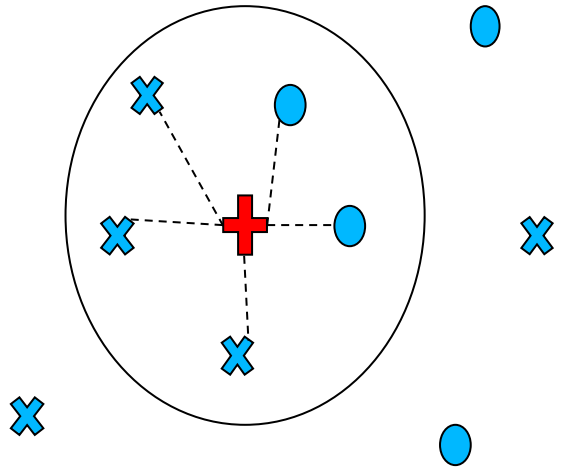


Standardization: Normalizes coordinates to prevent large-scale variables from dominating distance calculations.

Condensed Nearest Neighbor (CNN): A memory-saving technique that deletes redundant interior data points, keeping only the boundary prototypes necessary to maintain decision accuracy.

Using KNN in Python

```
>>> from sklearn.neighbors import NearestNeighbors
>>> import numpy as np
>>> X = np.array([[-1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1], [3, 2]])
>>> nbrs = NearestNeighbors(n_neighbors=2, algorithm='ball_tree').fit(X)
>>> distances, indices = nbrs.kneighbors(X)
>>> indices
array([[0, 1],
       [1, 0],
       [2, 1],
       [3, 4],
       [4, 3],
       [5, 4]]...)
>>> distances
array([[0.        , 1.        ],
       [0.        , 1.        ],
       [0.        , 1.41421356],
       [0.        , 1.        ],
       [0.        , 1.        ],
       [0.        , 1.41421356]])
```



Points	X1 (Acid Durability)	X2(strength)	Y=Classification
P1	7	7	BAD
P2	7	4	BAD
P3	3	4	GOOD
P4	1	4	GOOD

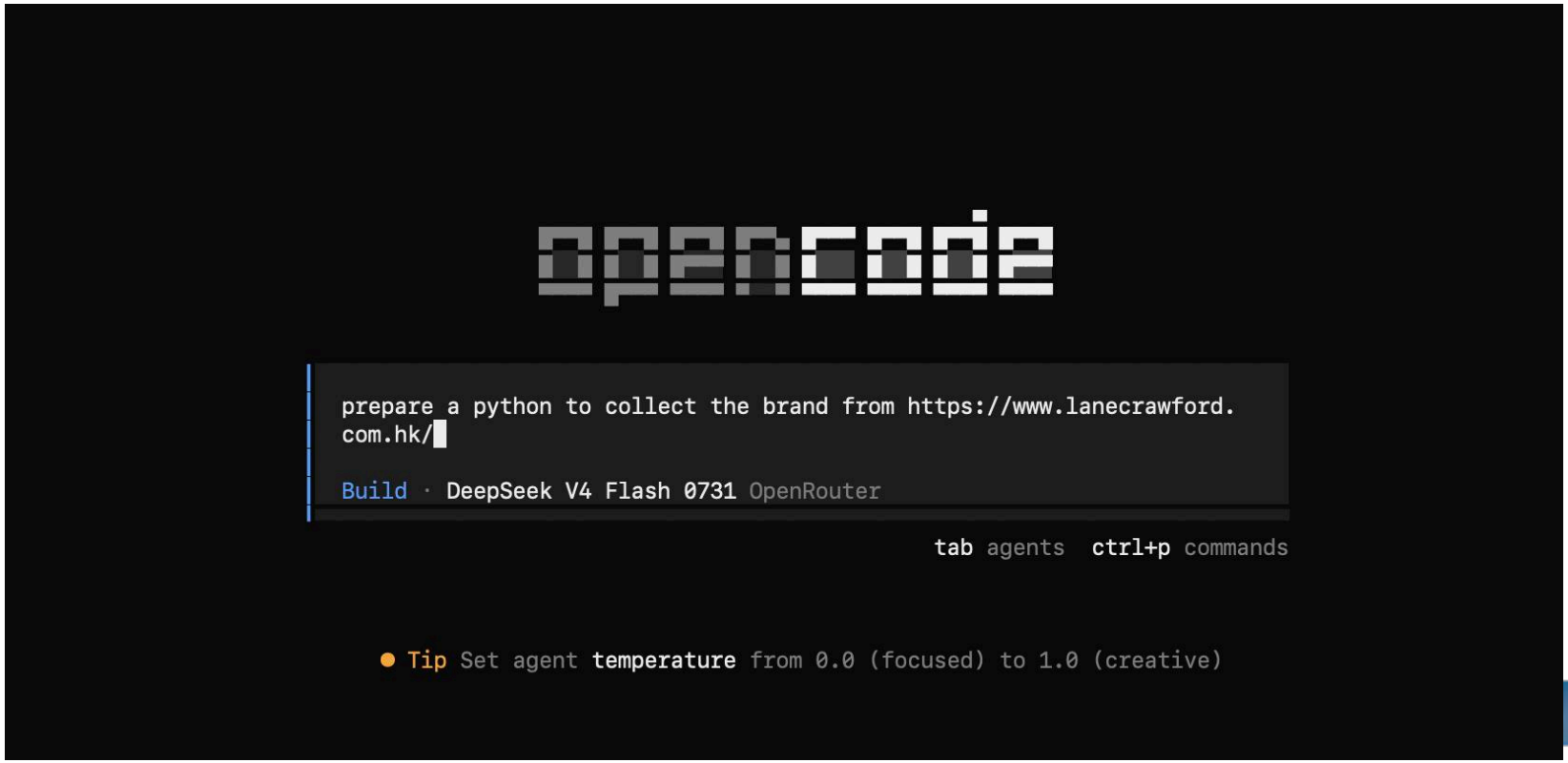
main.py

```
1  import numpy as np
2
3  score = np.array([10,3,2,4,5,10,8])
4  nationality = np.array([1,1,1,0,0,2,2])
5
6  # task 1: print the first element
7  # task 2: write a for loop to calculate the sum of all elements
8  # task 3: find the sum of the second and the last elements of score
9  # task 4: find the min, max and mean of score
10 # task 5: obtain the elements that are greater than 5
11 # task 6: obtain the elements that nationality is 1
12 # task 7: find the mean of the elements that nationality is 1
13 # task 8: write a function that will return the mean of the input data
14 # task 9: write a for loop to find the nationality with highest mean using the function
```



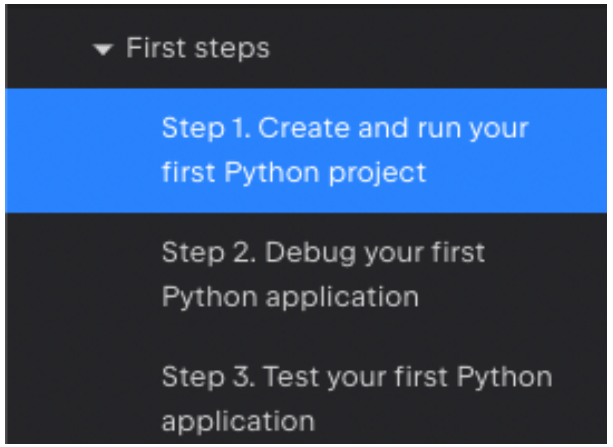
Demo: Scraping with OpenCode

- prepare a python to collect the brand from https://www.lanecrawford.com.hk/
- Visualize the brand



Readings

- Numpy: https://numpy.org/doc/stable/user/quickstart.html
- PyCharm: https://www.jetbrains.com/help/pycharm/creating-and-running-your-first-python-project.html#prerequisites
- Opencode: https://opencode.ai



Others

- Please bring your computer for the next lecture
- Install Opencode and PyCharm (professional version, free for students)
- Learn virtual environment and install library
- Run the example in this lecture



Questions?

Email: pakmingcheung@cuhk.edu.hk

